

**temperature=0 — значит, ответ  
всегда одинаковый?**



*Да, формально детерминирован. Но нет — на практике нет. Почему — сегодня.*

1/47

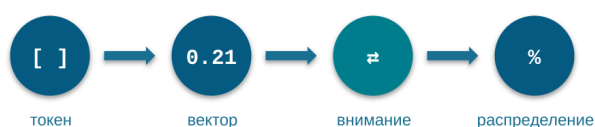
Простой вопрос на разогрев: если поставить temperature на ноль, ответ модели на один и тот же запрос будет каждый раз одинаковым? Интуиция подсказывает «да» — ноль случайности, выбор самого вероятного токена, что тут может пойти не так. И интуиция почти права. Почти — потому что «почти» на инженерной практике стоит денег: тесты, которые полагаются на побитовую воспроизводимость при  $T=0$ , время от времени необъяснимо падают, а причина не в вашем коде. Сегодня разберём, откуда берётся это «но нет» — и ещё несколько похожих мест, где интуиция про модели работает в девяти случаях из десяти, а в десятом ломается предсказуемым и объяснимым образом. Идём по конвейеру инференса от текста до ответа — и на каждой стадии посмотрим, где основанная на наблюдениях интуиция перестаёт быть надёжной картой.

## ЛЕКЦИЯ 2

# Как работают современные большие модели

02

Конвейер инференса — и границы, которые меняют инженерные решения



2/47

Это вторая лекция курса. Сегодня мы рассмотрим, как работает большая языковая модель изнутри: систематизируем устройство конвейера и разберём тонкости, которые напрямую влияют на инженерные решения. Кто работает с моделями ежедневно — освежит основы и узнает тонкости; кто встречается эти понятия впервые — узнает всё необходимое. Каркас лекции — конвейер инференса: путь одного запроса от текста до ответа. Текст режется на токены, токены превращаются в векторы, механизм внимания решает, какие части контекста важны, из распределения вероятностей выбирается очередной токен — и цикл повторяется. Четыре стадии — четыре раздела; пятый раздел — новый в этой версии курса — разбирает виды и размеры моделей: на чём они запускаются и что умеют; шестой раздел собирает конвейер целиком и подводит итоги. Двинемся по карте.

## Карта лекции — 7 разделов

**0** Введение — рамка и конвейер целиком

**1** Токенизация — как модель видит ваш текст

**2** Эмбединги — пространство смыслов и граница похожести

**3** Механизм внимания — что важно сейчас: роли, кэш, длинный контекст

**4** Сэмплинг — от распределения к токену: температура, детерминизм, невидимые токены

**5** Виды и размеры моделей — на чём модели запускаются, мультимодальность, ландшафт 2026

**6** Финал — сборка конвейера, итоги механизмов

3/47

Короткая карта маршрута, прежде чем погружаться. Лекция идёт вдоль конвейера инференса: порядок разделов повторяет порядок стадий, через которые проходит каждый ваш запрос. Раздел 1 — токенизация: как модель видит текст, почему «исправленный» strawberry ничего не доказывает, откуда берутся числа-чанки и glitch-токены. Раздел 2 — эмбединги: пространство смыслов, три разные вещи, которые называют одним словом «эмбеддинг», и граница, на которой семантическая похожесть перестаёт означать полезность. Раздел 3 — механизм внимания: почему промпт с ролью реально меняет ответ; откуда берётся экономика кэширования; что на самом деле умеет окно в миллион токенов. Раздел 4 — сэмплинг: температура, причина, по которой  $T=0$  не даёт детерминизма, структурированный вывод и невидимые токены рассуждения, которые вы оплачиваете. Раздел 5 — новый в этой версии курса: виды и размеры моделей — на чём запускается маленькая модель, а на чём гигант, что каждый класс умеет по мультимодальности, и ландшафт конкретных моделей на сентябрь 2026 года. Раздел 6 — финал: конвейер целиком и сводка разобранных механизмов. Держите карту в голове — она не даст потеряться в деталях.

## Объект сегодня — слой «модель» из четырёх слоёв Лекции 1



**Слой «модель»:** stateless-инференс — на вход данные, на выход предсказание, без памяти между вызовами.

**Сегодня:** разбираем, что происходит внутри этого инференса — и где его устройство меняет инженерные решения.

4/47

Объект сегодняшнего разговора — слой «модель». Лекция 1 дала слоистую картину: внизу модель, над ней цикл чата с системным промптом и историей, над чатом — агентный цикл, сверху — приложение. Модель там описана как stateless-инференс: на вход данные, на выход предсказание, без памяти между вызовами. Оттуда же у нас есть контекстное окно как ограничение, различие локальных и облачных моделей и три уровня причинности Пёрла — всё это сегодня считается известным и используется без повторного вывода. С этим описанием мы умеем говорить о том, где модель находится в большей системе. Но вопрос «что именно происходит внутри этой функции между запросом и ответом» Лекция 1 сознательно оставила чёрным ящиком. Сегодня мы этот ящик разбираем — и всё, что произойдёт на этой лекции, происходит внутри одного-единственного слоя, нижнего. Чат, агенты, RAG и приложения строятся поверх и обсуждаются в Лекции 3 и дальше. Мы идём за границами: в каких местах внутреннее устройство модели меняет то, как вы строите промпты, агентов и решения. Каждый механизм в лекции выбран потому, что у него есть наблюдаемое инженерное следствие — в стоимости, скорости, качестве или воспроизводимости.

## Цель лекции

«Рассмотреть, как работает языковая модель, — и разобраться с **важными деталями**, которые меняют то, как вы строите промпты, агентов и решения.»

почему исправленный strawberry ничего не доказывает

почему промпт с ролью реально меняет ответ

что на деле умеет окно 1M

почему  $T=0$  не даёт одинаковых ответов

сколько стоят невидимые токены рассуждения

маленькая модель или гигант — по какому критерию выбирать

чем заменить веру в бенчмарки

5/47

Цель лекции: рассмотреть, как работает языковая модель, — и разобраться с важными деталями, которые меняют то, как вы строите промпты, агентов и решения. Мы пройдём конвейер инференса от текста до ответа, и на каждой стадии — вместе с механизмом — покажем его границы. Мы покажем, где именно токенизация ломает арифметику и почему «исправленный» strawberry ничего не доказывает. Покажем механизм, из-за которого роль в промпте реально меняет ответ. Разберём, почему  $temperature=0$  не даёт воспроизводимости и сколько стоит её получить по-настоящему. Добавим к этому реальную цену невидимых токенов рассуждения, критерий выбора между маленькой моделью и гигантом — и то, чем заменить веру в бенчмарки при выборе модели. Семь строк под целью — по одной на каждый раздел, который сегодня пройдём: на каждую к концу лекции будет ответ через конкретный механизм, а не через интуицию. Если по ходу покажется, что какой-то механизм не влияет на практику, — ловите меня на слове: таких мы сегодня не разбираем.

## Поток данных в LLM — туда и обратно



6/47

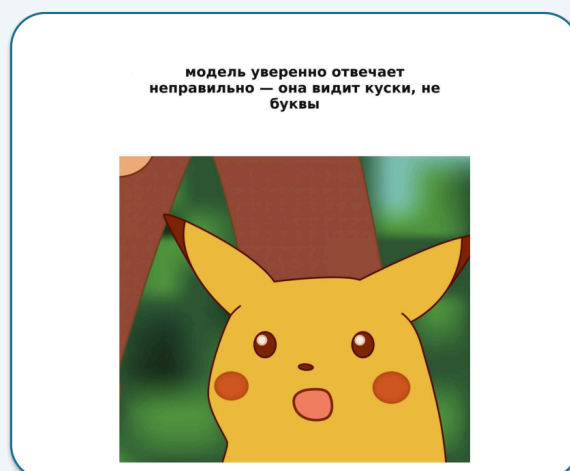
Это опорная схема всей лекции — конвейер инференса, путь одного запроса от текста до ответа. Посмотрим на него целиком, до первого погружения; дальше мы будем возвращаться к нему на входе в каждый раздел. Слева — ваш текст. Первое преобразование — токенизация: текст режется на токены, идентификаторы из словаря модели; это Раздел 1. Дальше каждый токен превращается в вектор — список чисел, с которыми нейросеть умеет считать; это Раздел 2. В центре — сама модель: механизм внимания решает, какие части контекста важны для следующего шага; это Раздел 3. На выходе модель выдаёт не ответ, а распределение вероятностей по всему словарю; из него выбирается один очередной токен, он приклеивается к контексту — и цикл повторяется; это Раздел 4. Последний шаг — выбранные токены собираются обратно в текст. Главное наблюдение: слова существуют только на границах конвейера — на входе и на выходе. Всё внутри — операции над векторами. И обратите внимание на петлю в схеме: токены генерируются по одному, каждый выбранный токен добавляется ко входу — и конвейер прокручивается заново; ответ строится последовательными оборотами этой петли. Эта ось — не только про сегодня. Все последующие лекции курса лягут на неё же: RAG — это управление тем, что попадает в контекст до конвейера; агенты — цикл вокруг конвейера; оптимизация стоимости — экономика конвейера. Сегодняшняя лекция движется по этой же оси — каждый раздел раскрывает свой участок конвейера вместе с границами применимости.

# Раздел 1

## Токенизация

«Как модель видит ваш текст»

3 разбора · 3 провала



7/47

Входим в первую стадию конвейера — токенизацию. Это самый ближний к вам слой: именно с ним сталкивается текст, как только вы нажимаете «отправить». Текст не попадает в модель ни буквами, ни словами — он режется на токены, статистически частые подпоследовательности из словаря модели. Определение токена вы знаете, поэтому базу мы пройдем быстро и точно — она нужна как общий фундамент. Основное время раздела уйдет на второй этаж: места, где частотная нарезка расходится со структурой ваших данных. Посмотрим на диагноз 2026 года по классическому мему про подсчет букв: что именно доказывает модель, которая проходит strawberry и ошибается на cranberry. Увидим, что токенизатор делает с числами и кодом и почему это ломает арифметику. Познакомимся с glitch-токенами — недообученными углами словаря, которые никуда не делись и в современных моделях. И закроем раздел экономикой: сколько стоит один и тот же текст на разных языках и что с этим делать при настройке лимитов. Общий принцип раздела: у каждой странности токенизации есть механизм, а у механизма — проверяемое инженерное следствие.

## Токен — id из словаря модели: не буква и не слово

cat → [cat] → 1 токен

tokenization → [token][ization] → 2 токена

клубника → [к][луб][ника] → 3 токена (o200k\_base)

**«В среднем: 1 токен ≈ 4 символа EN ≈ 2 символа RU»**

*Словарь и модель — два разных артефакта: словарь строится до обучения модели, отдельным алгоритмом на своём корпусе.*

8/47

Точное определение. Токен — это идентификатор, целое число из словаря модели; словарь зафиксирован при обучении и не меняется в момент использования. Токен — не буква и не слово, а статистически частая подпоследовательность символов, выученная на корпусе. Частое английское `cat` попадает в словарь целиком; `tokenization` распадается на два токена; `клубника` — на три в том же словаре o200k\_base. Размер словаря у современных моделей — порядка сотен тысяч записей: около 200 тысяч у семейства GPT-4o и новее, около 100 тысяч у предыдущих поколений, 128 256 у Llama 3 и новее. Рабочий ориентир стоимости: один токен — примерно четыре символа английского и примерно два символа русского текста. Одно уточнение, которое часто теряется: словарь и модель — два разных артефакта. Словарь строится отдельным алгоритмом на своём корпусе до обучения модели; модель потом учится работать с этим словарём на своём корпусе. Эта развязка — источник целого класса эффектов, к которым мы вернёмся в этом разделе. И ответ на резонный вопрос «почему не работать посимвольно»: экономика длины. Посимвольное представление удлиняет вход в три-пять раз, а стоимость внимания растёт с длиной квадратично — вчетверо длиннее вход означает в шестнадцать раз дороже слой внимания. Токенизация со всеми её артефактами — осознанно купленный компромисс, а не недосмотр, который «починят в следующей версии».

## BPE — компромисс между алфавитом и словарём

Не все буквы (длинно) и не все слова (незнакомое выпадает) — частые подпоследовательности.



Разные вендоры режут один и тот же текст по-разному: Claude, GPT, Gemini — свои словари и правила.

9/47

Алгоритм построения словарей большинства современных LLM — BPE, байт-парное кодирование. Это компромисс между двумя крайностями. Словарь из отдельных символов представит любой текст, но последовательности получаются длинными. Словарь из целых слов даёт короткие последовательности, но любое незнакомое слово — опечатка, неологизм, имя — выпадает. BPE сидит посередине: начинаем с алфавита, итеративно объединяем самые частые пары соседних единиц; после многих итераций в словаре есть и одиночные символы, и частые слоги, и целые высокочастотные слова. Ключевая инженерная деталь: словарь строится один раз, до обучения модели. На корпусе запускают BPE, фиксируют словарь и правила слияния — и всё; на инференсе токенизация — это выборка по готовой таблице за миллисекунды. Отсюда два следствия, которые пригодятся ниже: токенизатор режет по частотной статистике своего корпуса, и эта статистика может не совпадать со структурой вашей задачи; а раз словарь и модель обучаются на разных корпусах — в словаре могут остаться записи, которые модель почти не видела. Практически важно и другое: разные вендоры считают один и тот же текст по-разному — свои словари, таблицы слияния, обработка пробелов. Один документ у разных провайдеров даст разное число токенов, а значит, разную стоимость и разное заполнение окна. При переносе нагрузки между провайдерами пересчитывайте токен-бюджет: для моделей OpenAI токенизация эмулируется локально библиотекой tiktoken, для открытых моделей — библиотекой tokenizers; токенизатор Claude закрыт, но фактический расход виден в счётчиках любого ответа API.

## Гонка патчей: strawberry починили в апреле, cranberry — только в июле

### Механизм — слепота к буквам

strawberry → [st][raw][berry] Модель видит 3 токена, не 10 букв.

|                                     |  |
|-------------------------------------|--|
| GPT-5.2: 2 r в strawberry           |  |
| GPT-5.5: strawberry ✓ / cranberry X |  |
| GPT-5.6: cranberry ✓                |  |
| StrawberryBench: всё ещё 60%        |  |

### GPT-5.2 · дек 2025

strawberry X — «в strawberry две r»

### GPT-5.5 · апр 2026

strawberry ✓ / cranberry X — «две r» вместо трёх

### GPT-5.6 · июль 2026

cranberry ✓ — очередной вирусный кейс запатчен

### StrawberryBench

847 вопросов, 7 уровней сложности — системная проверка вместо вирусного вопроса

**«Рванный интеллект» (jagged intelligence): уровень навыка задаётся данными обучения, а не «общим умом» — модель берёт олимпийское золото и проваливает подсчёт букв.**

**Что делать:** тестируйте «cranberry своей предметки» — редкие случаи, на которых никто не хайпил; вирусное прохождение ≠ навык.

10/47

Мем «сколько r в strawberry» вы знаете; вероятно, знаете и то, что современные модели отвечают на него правильно. Вывод «значит, научились считать буквы» — ложный, и вот почему. Механизм — слепота к буквам. Слово приходит в модель как три токена, а не десять букв. Внутри выученного вектора токена нет поля «содержит r на такой-то позиции» — там статистика контекстов. Посимвольный подсчёт требует рассуждения поверх токенизированного представления — разворачивания слова по буквам или вызова кода; один прямой проход к надёжному ответу не приводит. Это структурное свойство конвейера. Теперь смотрите на гонку патчей 2026 года. GPT-5.2 в декабре 2025-го всё ещё отвечал «две r» в strawberry. GPT-5.5, выпущенный в апреле 2026-го, проходит strawberry — но на вопрос «сколько r в cranberry» отвечал «две»; правильно — три. И только GPT-5.6 в июле 2026-го починил уже и cranberry — проверьте актуальную модель прямо сейчас. Обратите внимание на закономерность: каждый вирусный кейс патчится точно и по очереди, а не единым улучшением навыка — иначе фикс strawberry сразу закрыл бы и cranberry. Для систематической проверки класса задачи появился StrawberryBench — 847 вопросов в семи уровнях сложности; именно на нём, а не на одном вирусном слове, видно, что посимвольный счёт остаётся проблемным местом. У явления есть имя: «рванный интеллект» — профиль способностей с резкими провалами рядом с пиками. Причина в том, что уровень навыка задаётся данными обучения, а не «общим умом»: где в данных навык представлен богато — там пик, где представление слова скрывает нужную структуру — там провал. Та же модель решает олимпиадную математику и проваливает подсчёт букв; на умножении GPT-4 без инструментов давал около 59% на трёхзначных числах, 4% на четырёхзначных и 0% на пятизначных. Урок переносим в практику: если модель прошла ваш тест — проверьте её на «cranberry вашей предметной области», структурно аналогичном, но

незнаменитом примере, на котором никто не хайпил: вирусное прохождение — не навык. А для посимвольных операций и арифметики — внешний инструмент, не прямой проход.

## Токенизатор режет по частоте, а не по структуре

Числа и код — самые частые «нетекстовые» входы: от их нарезки зависят арифметика модели и ваш бюджет токенов.

### Числа

1000000 → [100][000][0]

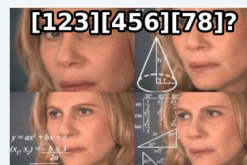
Чанки слева направо — а математические разряды считаются справа налево: границы не совпадают, сложение в столбик ломается.

Нарезка справа налево улучшает арифметику; задача-специфичные схемы — **до +33% точности** к стандартной нарезке.

### Код

GPT-2: 16 токенов на отступ 4-го уровня.

GPT-4: пробелы группами — словарь чинится, но не под все задачи сразу.



### Что делать:

Разделители разрядов («1 234 567»)

Вычисления — в инструмент

Единообразные отступы

Готовые чат-продукты уже сами уводят счёт во встроенные инструменты (code interpreter); звать инструмент самому — нестандартные случаи и свои приложения поверх API.

11/47

Зачем инженеру знать, как режутся числа и код: это самые частые «нетекстовые» входы, и от их нарезки напрямую зависят арифметика модели и ваш бюджет токенов на код. Два компактных примера того, как частотная нарезка расходится со структурой данных. Числа. Токенизатор cl100k\_base стандартизировал нарезку чисел чанками по три цифры слева направо: миллион превращается в группы, границы которых не совпадают с разрядами. Человек читает разряды справа налево — тысячи, миллионы; модель получает нерегулярные блоки — и это прямой источник части арифметических ошибок. Исследования подтверждают диагноз с обеих сторон: принудительная нарезка справа налево заметно улучшает численное рассуждение, а задача-специфичные схемы токенизации чисел давали до плюс тридцати трёх процентов точности на арифметике с большими числами относительно стандартной нарезки. Код. GPT-2 кодировал каждый пробел отступа отдельным токеном: строка на четвёртом уровне вложенности тратила шестнадцать токенов только на отступ. Токенизаторы поколения GPT-4 группируют пробелы в единые токены, целясь именно в Python-стиль. Это редкий случай, когда проблему токенизации починили изменением словаря, — полезный контраст к strawberry: словарь можно оптимизировать под частый класс входов, но нельзя совместить нарезку со структурой всех задач сразу. Четыре приёма на вынос. Значимые числа записывайте с разделителями разрядов — «1 234 567» вместо сплошной строки: разделители выравнивают границы токенов по разрядам. Любую арифметику важнее прикидки выносите в инструмент — модели прекрасно пишут код для вычислений, которые не могут выполнить сами. Для кода — единообразное форматирование и оценка бюджета по измерению на реальных файлах: средний ориентир «четыре символа на токен» для кода систематически врёт. И важная поправка для практики: в готовых чат-продуктах — ChatGPT, Claude в

веб-интерфейсе — счёт и арифметика уже автоматически уходят во встроенный инструмент вроде code interpreter, минуя прямой проход модели; то, что мы сейчас разобрали, становится критичным именно тогда, когда вы вызываете модель через API напрямую — в своих приложениях и агентах, где такого автоматического роутинга нет, и в нестандартных случаях, которые готовый продукт не распознал как «нужен инструмент».

## Порядка 4% словаря — glitch-токены

### SolidGoldMagikarp (2023)

Юзернейм с Reddit в словаре GPT — модель не могла его повторить (имя токена — отсылка к покемону Magikarp).



**Механизм:** корпус словаря ≠ корпус модели → эмбединг токена остаётся у случайной инициализации.

### На что влияет

- Сбои парсинга экзотических строк — редкие идентификаторы, обфусцированный текст, необычный Unicode
- Риски в проде — логи, user-generated ID, произвольный пользовательский ввод
- Необъяснимые ответы «невыпадет» без ошибки в коде

### GlitchMiner (AAAI 2026)

**порядка 4% словаря** по одной из оценок;

воспроизводится в открытых семействах Llama, Qwen, Gemma, Phi-3, Mistral.

### Что на практике:

Необъяснимое поведение на экзотических строках (редкие идентификаторы, обфусцированный текст, необычный Unicode)?  
Гипотеза: glitch-токен. Проверка: замените строку плейсхолдером.

В продуктах, принимающих произвольный ввод, — нормализация и санитизация входа до модели.

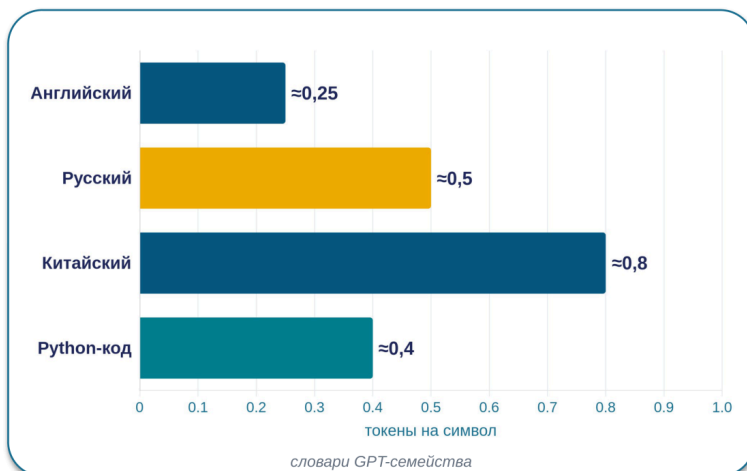
*Системная особенность конвейера, не баг версии — масштабом не чинится.*

12/47

В январе 2023 года исследователи, изучая кластеры в пространстве эмбедингов GPT-моделей, наткнулись на группу странных записей словаря — строки вроде SolidGoldMagikarp: юзернеймы с Reddit, попавшие в корпус токенизатора при сборе данных. Модели вели себя на этих токенах аномально: не могли их повторить, отвечали невыпадет, уходили от темы. Механизм коротко: строка может быть частой в корпусе, на котором строился словарь, и получить собственный токен — но почти не встречается в корпусе, на котором потом обучалась модель; эмбединг такого токена остаётся близким к случайной инициализации и «ничего не значит» в выученной геометрии. Данные 2026 года говорят: это не курьёз эпохи GPT-3, а системное свойство. По одной из оценок, порядка четырёх процентов записей словаря протестированных моделей — glitch-токены, а фреймворк GlitchMiner находит их градиентным поиском в десяти открытых LLM семействах Llama, Qwen, Gemma, Phi-3 и Mistral. Проблема воспроизводится во всех проверенных семействах, потому что словарь строится отдельно от модели; масштабом такое не чинится. Теперь — на что это реально влияет в инженерной практике, и это важнее самого механизма. Первое: сбои парсинга на экзотических строках — модель необъяснимо теряет тему разговора или отказывает именно на конкретном входе, а не в целом на классе задач. Второе: прод-риски — любая система, принимающая произвольный пользовательский ввод, работает с потенциальным источником glitch-токенов: логи, автоматически сгенерированные ID, обфусцированный текст, редкие Unicode-последовательности. Третье: диагностика — если поведение необъяснимо именно на конкретном входе, среди гипотез должна быть «на входе glitch-токен»; проверяется за минуту заменой подозрительной строки на плейсхолдер — поведение выправились, виновник найден. Четвёртое, продуктовое: нормализуйте и санитизируйте вход до подачи в модель — это

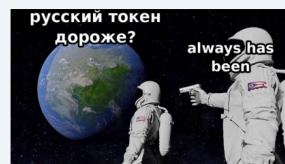
отсекает не только glitch-токены, но и целый класс смежных сюрпризов токенизации.

## Русский текст стоит $\approx 2\times$ дороже английского



### Переход на o200k\_base:

примерно **–35%** нелатинским языкам — разрыв сокращается, но не исчезает.



### Что делать:

- Калибруйте лимиты в токенах на своём языке: фрагменты для поиска, `max_tokens`, бюджет окна.
- Пакетная обработка больших объёмов — оцените перевод на английский ( $\approx 2\times$  дешевле); в интерактиве разница того не стоит.

13/47

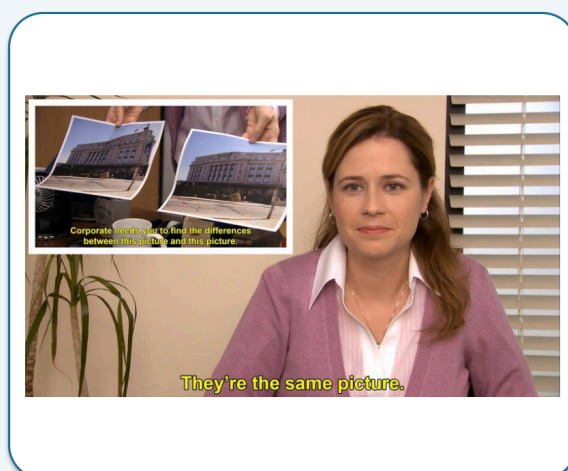
Один и тот же по смыслу текст стоит разное число токенов в зависимости от языка — прямое следствие того, что BPE-словарь учится на корпусе с доминирующим английским. Ориентиры для токенизаторов GPT-семейства: английский — около 0.25 токена на символ, русский — около 0.5, китайский — около 0.8, Python-код — около 0.4. Итого запрос на русском обходится примерно вдвое дороже английского, с разбросом от полутора до двух с половиной раз, и контекстное окно расходуется вдвое быстрее: документ в 80 тысяч символов — это примерно 20 тысяч токенов на английском и 40 тысяч на русском. Динамика 2024–2026 годов: переход OpenAI на o200k\_base снизил удельную стоимость нелатинских языков примерно на 35% — разрыв сокращается, но не исчезает: английский остаётся статистическим ядром корпусов. Для моделей с увеличенной долей русского в словаре — YandexGPT, GigaChat — разрыв меньше; это один из рациональных аргументов в их пользу для массовых русскоязычных нагрузок. Правило прежнее: при пакетной обработке больших объёмов оцените, допускает ли задача перевод на английский; в интерактивной работе разница обычно не оправдывает потери удобства. Языковой коэффициент протекает и туда, где о нём забывают. Разбиение документов на фрагменты для поиска настраивается в токенах — фрагмент «в 512 токенов» на русском вмещает вдвое меньше смысла, и пороги из англоязычных руководств для русской базы систематически малы. Лимит `max_tokens` — та же история: русский ответ длиннее в токенах, и обрезки на полуслове случаются чаще. Любой числовой параметр в токенах калибруйте на своём языке и своих данных.

# Раздел 2

## Эмбединги

«Пространство смыслов — и граница  
похожести»

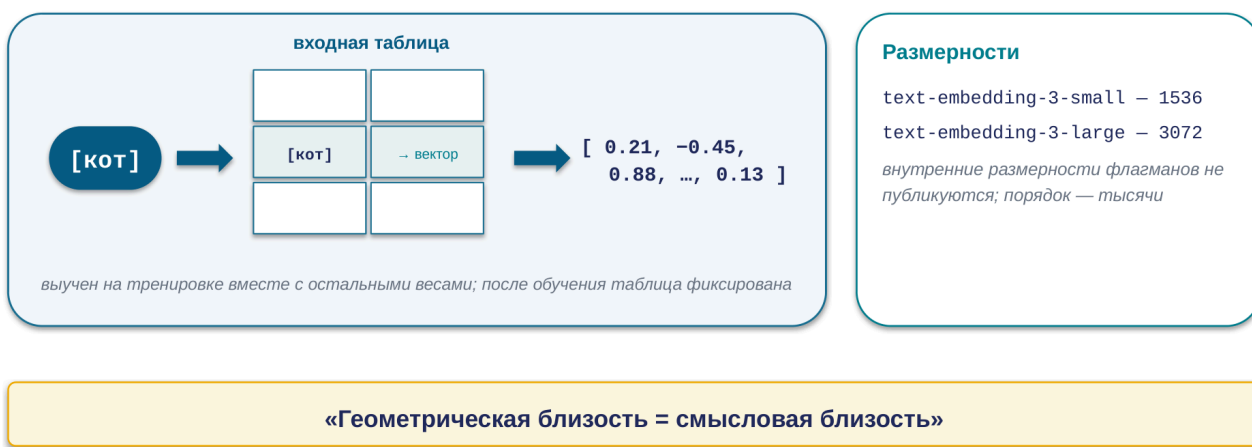
4 разбора · 1 провал



14/47

Первая стадия пройдена: текст стал последовательностью идентификаторов из словаря. Но с самим числом нейросеть содержательно работать не может — между номерами токенов нет осмысленной арифметики. Вторая стадия конвейера превращает идентификаторы в представление, с которым можно считать, — векторы. План раздела такой. Сначала зафиксируем, что такое эмбединг: выборка вектора из входной таблицы модели — быстро, это база. Затем — главный систематизирующий ход раздела: слово «эмбединг» в инженерном обиходе означает три разные сущности, и их нередко смешивают; разложим три жизни термина аккуратно — и увидим, что эмбединги работают не только внутри инференса, но и как самостоятельный инструмент поиска. Дальше — пространство смыслов: как выучиваются координаты, почему близкие по смыслу тексты лежат рядом и как это измеряется косинусным сходством. После этого — единственный, но важный провал раздела: граница похожести, случай, когда высокое сходство приносит практически противоположный по смыслу документ, и что с этим делают в боевых системах поиска. И в конце соберём раздел: почему эмбединги — фундамент того, что модель «понимает» переформулировки, и что из этого следует для систем поиска и RAG, к которым мы придём в Лекции 3.

## Каждому токenu — вектор из входной таблицы модели



**Что делать:** опечатка или другой регистр — уже другой токен и другой вектор; нормализуйте вход до эмбединга.

15/47

Токен — это идентификатор из словаря, но с числом 48 213 нейросеть содержательно работать не может: между номерами токенов нет осмысленной арифметики. Эмбединг, векторное представление, — сопоставленный каждому токenu словаря вектор фиксированной длины, список чисел с плавающей точкой, выученный на этапе обучения вместе с остальными весами. После обучения входная таблица «токен → вектор» зафиксирована; на инференсе модель делает выборку: получает идентификатор, забирает вектор, передаёт дальше. Главное свойство выученного пространства: геометрическая близость соответствует смысловой. «Кот» близко к «собаке», «SSL» близко к «HTTPS» — не потому, что кто-то это разметил, а потому, что эти слова встречались в похожих контекстах обучающего корпуса. Близость в этом пространстве — статистическое отражение употребления, а не семантический справочник. Размерность — гиперпараметр: у публичных embedding-моделей OpenAI — 1536 и 3072 измерения; внутренние размерности флагманских LLM не публикуются, порядок — тысячи. И одна оговорка для тех, кто будет работать с векторами руками. Представьте трёхмерное пространство, где близкие по употреблению слова — рядом, и растяните число измерений до тысяч. Интуиции близости в основном переносятся, но в пространствах высокой размерности действует концентрация меры: случайные точки оказываются примерно на одном расстоянии друг от друга, и «сырые» дистанции сжимаются в узкий диапазон. Поэтому абсолютные значения сходств малоинформативны сами по себе — информативны сравнения и распределения значений в вашей задаче. Это первая причина, по которой универсальных порогов сходства не существует; вторая, содержательная, — впереди.

## Эмбединги — не только внутренность инференса, но и инструмент поиска

Когда вы строите поиск или RAG, вы пользуетесь третьей жизнью термина «эмбединг» — отдельной *embedding-моделью*.

### 1

#### Входная таблица

*внутри инференса*

Статическая: вектор [кот] один и тот же в любом предложении. Выборка по id, контекста ещё нет.

### 2

#### Представление данных внутри модели

*внутри инференса*

Векторы, которые модель пересчитывает по ходу чтения контекста — после слов внимания. Именно они несут «понимание» модели.

### 3

#### Векторы для поиска

*самостоятельный инструмент*

Вектор целого текста от отдельной *embedding-модели* — не внутренности вашего чат-LLM. Свой продукт, своё обучение, свои лидерборды.

*это и есть ваш поиск/RAG*

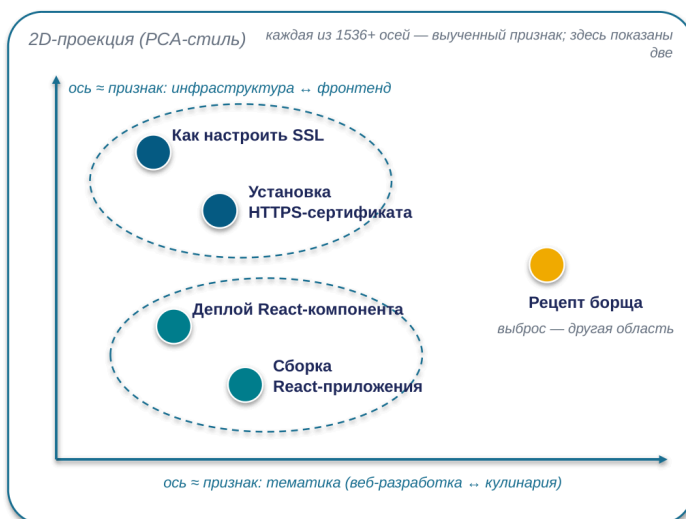
Обновили чат-LLM → переиндексировать базу НЕ надо: индекс живёт в координатах *embedding-модели*.

16/47

Эмбединги нужны не только для инференса — это ещё и самостоятельный инструмент поиска. Слово «эмбединг» в инженерном обиходе обозначает три разные сущности; разложим три жизни термина и посмотрим, какой из них вы пользуетесь, когда строите поиск. Жизнь первая: входная таблица — статическая таблица «токен → вектор» на входе LLM. Вектор токена «кот» в ней один и тот же в любом предложении. Жизнь вторая: представление данных внутри модели. Последовательность входных векторов проходит через десятки слоёв внимания, и на каждом слое вектор каждой позиции обновляется с учётом окружения — на выходе вектор позиции «кот» уже не равен табличному, в него вмешан контекст; это векторы, которые модель пересчитывает по ходу чтения контекста. Именно эти представления несут «понимание» модели; входная таблица — лишь стартовая точка. Обе эти жизни — внутренности инференса. А третья жизнь — ваш рабочий инструмент: когда вы вызываете *embedding-API* для поиска или RAG, вы получаете вектор целого текста от отдельной модели, специально обученной под поиск и сравнение. Это не внутренности вашего чат-LLM: *embedding-модель* — самостоятельный продукт, и чат-модель с *embedding-моделью* могут быть от разных вендоров — это нормальная практика. Правило разграничения: речь про стоимость и окно — это токены и входная таблица; про «что модель поняла» — представление данных внутри модели; про поиск и векторные базы — выходные эмбединги отдельной модели. И типовые ошибки склейки, ради которых различие вводится. «У нас подписка на чат-модель — зачем платить за *embedding-модель*?» — затем, что чат-модель векторов для поиска не отдаёт. «Возьмём *embedding-модель* того же вендора — они совместимее» — совместимости здесь нет как категории: векторы сравниваются только друг с другом внутри одной *embedding-модели*, LLM их вовсе не видит. И зеркально: «обновили LLM — надо переиндексировать базу» — не надо; переиндексация

нужна ровно тогда, когда меняется сама embedding-модель.

## Близкие по смыслу токены лежат рядом — в сотнях-тысячах измерений



### Размерность

Публичные embedding-модели: **1536–3072** измерения; внутренние у флагманов — порядка тысяч.

### Обучение

Координаты не задаются вручную: похожие контексты употребления → близкие векторы.

### Проекция

Увидеть пространство можно только через PCA/t-SNE — 2D-картинка теряет часть структуры.

Что делать: близость измерима расстоянием — фильтрация и кластеризация без разметки и без LLM возможны прямо на векторах, дёшево.

17/47

Как устроено пространство, в котором живут векторы. Каждая точка — токен или текст; координаты — те самые сотни или тысячи чисел. Смысл измерений не задаётся вручную — это направления, которые модель выучила из статистики; постфактум многие из них читаются как признаки: тематика, формальность, домен. Цель обучения — устроить пространство так, чтобы единицы, встречавшиеся в похожих контекстах, лежали близко. «Как настроить SSL» и «Установка HTTPS-сертификата» окружены одними и теми же словами — «сертификат», «сервер», «домен» — и их векторы получаются близкими; «Деплой React-компонента» и «Сборка React-приложения» соседствуют со словами про фронтенд и сборку — и тоже оказываются рядом; а «Рецепт борща» живёт в совсем другой области пространства — одинокий выброс среди технических текстов. Структура возникает не из правил, а из миллионов примеров употребления. На картинке слева — двумерная проекция: два смысловых кластера и выброс — те же пять текстов встретятся дальше в таблице попарных сходств. Оси проекции подписаны как признаки: одна разводит веб-тематику и кулинарию, другая — инфраструктуру и фронтенд; каждая из полутора с лишним тысяч осей реального пространства — такой же выученный признак, здесь показаны две. Помните, что это упрощение: реальная проекция получается из пространства в тысячи измерений алгоритмами вроде метода главных компонент или t-SNE, которые выбирают две оси с максимумом различий. Любая плоская картинка теряет часть многомерной структуры — она годится для интуиции, не для выводов. И повторим оговорку, потому что она стоит денег на практике: в пространствах высокой размерности «сырые» расстояния сжимаются в узкий диапазон, и абсолютное значение сходства само по себе мало о чём говорит. Работают сравнения — «этот документ ближе того» — и распределения значений в вашей конкретной задаче. Универсального порога «выше 0.8 — значит похоже»

не существует; порог калибруется эмпирически на ваших данных. Дальше посмотрим, как эта геометрия работает на целых предложениях — и где у неё проходит граница.

**Высокое сходство — «об одном и том же», не «с одинаковым смыслом»**

| cosine similarity           | SSL  | HTTPS | React-к. | React-п. | Борщ |
|-----------------------------|------|-------|----------|----------|------|
| Как настроить SSL           | 1,00 | 0,85  | 0,18     | 0,20     | 0,08 |
| Установка HTTPS-сертификата | 0,85 | 1,00  | 0,22     | 0,19     | 0,07 |
| Деплой React-компонента     | 0,18 | 0,22  | 1,00     | 0,78     | 0,12 |
| Сборка React-приложения     | 0,20 | 0,19  | 0,78     | 1,00     | 0,10 |
| Рецепт борща                | 0,08 | 0,07  | 0,12     | 0,10     | 1,00 |

«Как настроить SSL» ↔

«Как отключить SSL»

Очень высокое сходство — противоположный практический смысл.

шкала: 0 — светлое · 1 — тёмное

«похожи» — не факт что  
«об одном»

Similarity — сигнал кандидатов; релевантность — отдельная задача: реранкер, гибриды, фильтры.

18/47

Сначала — что похожесть работает. Пять коротких текстов, парная таблица косинусных сходств на современной embedding-модели: синонимичные по задаче пары — «настроить SSL» и «установка HTTPS-сертификата» — около 0.85; тематически близкие React-тексты — около 0.78; борщ против любого технического — 0.05–0.15. Запрос «клубника» находит документы про strawberry и землянику без таблицы синонимов — это то, что эмбединги дают поверх полнотекстового поиска. Теперь граница — и это новый материал даже для тех, кто активно строит семантический поиск. Высокое косинусное сходство означает «об одном и том же», а не «об одном и том же с одинаковым смыслом». Пара «как настроить SSL» и «как отключить SSL» получает очень высокое сходство: одинаковая тема, лексика, синтаксическая рамка — а практический смысл противоположный. Эмбединг усредняет контекстную статистику всего текста; короткое отрицание или антонимичный глагол сдвигает вектор слабо. В проде это выглядит болезненно: пользователь спрашивает, как включить проверку сертификатов, поиск уверенно приносит статью, как её отключить, LLM в RAG-конвейере добросовестно пересказывает — и система с высокими метриками похожести выдаёт вредный совет. Инженерные ответы известны: реранкер — вторая модель, оценивающая пару «запрос-документ» целиком; гибриды с полнотекстовым поиском для точных терминов; фильтры и метаданные для направленных атрибутов. И приём диагностики: соберите проверочный набор пар «запрос → правильный документ» с намеренными ловушками — вкл/выкл, до/после, разные версии — и посмотрите, что стоит на первых позициях. Устройство RAG-конвейера целиком — Лекция 3; отсюда забираем принцип: косинусное сходство — про тематическую близость, релевантность — отдельная задача.

## Эмбединги — фундамент понимания: модель работает с векторами, не строками



### Перефразирования и синонимы

«Как настроить SSL» и «Установка HTTPS-сертификата» — близкие векторы → модель отвечает одинаково; то же с «авто» и «машина».

### Межъязыковая близость

«клубника» и strawberry — близкие векторы → ответ корректен независимо от языка запроса.

### Семантическая близость на уровне предложений — основа «понимания» переформулировок.

**Что делать:** одна embedding-модель и индекс обслуживают поиск, кластеризацию и RAG сразу — смена модели означает переиндексацию всего хранилища; выбирайте как инфраструктурное решение.

*Выбор embedding-модели под задачу (MTEB, матричные представления) — материал для самостоятельного чтения.*

19/47

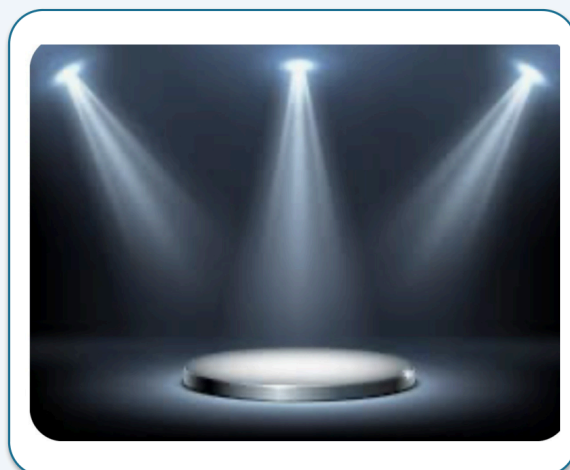
Соберём раздел. Модель работает не со словами и не со строками — она работает с векторами; слова существуют только на границах. На входе текст через токенизацию превращается в идентификаторы, через входную таблицу — в векторы; на выходе внутренний вектор через распределение и выбор токена возвращается в текст. Между человеческим вводом и человеческим выводом — путь полностью векторный. Это объясняет то, что мы интуитивно называем «пониманием». Когда вы спрашиваете «как настроить SSL», а потом «установка HTTPS-сертификата», модель отвечает похоже, потому что оба запроса попадают в близкие точки пространства. То же с синонимами и с межъязыковой близостью: «клубника» и strawberry — близкие векторы, и ответ корректен независимо от языка запроса. Практическая сторона: на одних и тех же векторах стоят три стандартных применения — поиск похожих, кластеризация без разметки и семантический поиск с RAG. Одна embedding-модель и один индекс обслуживают сразу несколько функций продукта, поэтому выбор embedding-модели — инфраструктурное решение, а смена — дорогая: означает переиндексацию всего хранилища. В боевых системах семантический поиск почти никогда не живёт один: рабочий стандарт — гибрид из полнотекстового индекса для точных совпадений, векторного для понятийной близости и реранкера сверху — прямое следствие границы похожести, которую мы только что разобрали. И помните: обратной операции у эмбединга нет — из вектора текст не восстанавливается, это одностороннее сжатие смысла. Как выбирать embedding-модель под язык и бюджет размерности — оставляю вам для самостоятельного чтения: в методичке это глава, часть 1 — раздел об embedding-моделях 2026.

# Раздел 3

## Механизм внимания

*«Как модель решает, на что опереться в контексте — и что из этого следует для ролей, кэша и длинных окон»*

4 разбора · 2 провала



20/47

Две стадии конвейера позади. Токенизация: текст разрезан на токены — идентификаторы из словаря, зафиксированного до обучения. Эмбеддинги: каждый идентификатор получил выученный вектор, и близость векторов отражает близость употребления. Третья стадия — механизм внимания, самая насыщенная часть лекции. На входе — последовательность векторов; задача стадии — для каждого шага генерации решить, на какие части контекста модель опирается сильнее всего. Из этого механизма напрямую вырастают вещи, с которыми вы работаете каждый день: почему история чата кешируется и когда кэш перестает срабатывать; как устроена экономика кэширования промптов; почему промпт с ролью реально меняет ответ, но не делает ответ более точным; что на самом деле означает строка «окно 1 миллион токенов» в карточке модели — и где это обещание ломается. Порядок движения: сначала матрица внимания и точное определение с тремя проекциями Query/Key/Value на нашем примере; дальше — KV-cache и две фазы генерации; экономика кэширования промптов с живым кейсом; затем рабочий пример и полный механизм эффекта роли; гонка контекстных окон на сентябрь 2026 года; и в финале раздела — честный ответ на вопрос, сколько из миллионного окна модель действительно способна использовать для рассуждения.

## Внимание — это сверка каждого токена со всеми остальными

Каждый токен «смотрит» на все остальные одновременно. На каждом шаге —  $N \times N$  связей.

| вес        | Кот | съел | мышь | потому что | он  | был | голоден |
|------------|-----|------|------|------------|-----|-----|---------|
| Кот        | 1,0 | 0,3  | 0,2  | 0,1        | 0,1 | 0,1 | 0,0     |
| съел       | 0,4 | 1,0  | 0,5  | 0,1        | 0,1 | 0,1 | 0,1     |
| мышь       | 0,2 | 0,4  | 1,0  | 0,1        | 0,1 | 0,0 | 0,1     |
| потому что | 0,1 | 0,2  | 0,2  | 1,0        | 0,3 | 0,2 | 0,2     |
| он         | 0,7 | 0,1  | 0,2  | 0,2        | 1,0 | 0,3 | 0,4     |
| был        | 0,1 | 0,2  | 0,2  | 0,3        | 0,4 | 1,0 | 0,5     |
| голоден    | 0,1 | 0,2  | 0,3  | 0,3        | 0,4 | 0,5 | 1,0     |

По строке «он»: наибольший вес — на «Кот» (мужской род). Статистическая связь, выученная на корпусе.

В декодере токен видит только предыдущие — показана полная сверка для наглядности.

### Размерность

$N \times N$ , где  $N$  — длина контекста. Удвоение контекста — учетверение вычислений внимания.

### На каждом шаге

Распределение весов пересчитывается заново на каждом шаге генерации.

### Многоголовость

В каждом слое — десятки параллельных «голов» (типично 32–128); каждая ловит свой тип связей: грамматика, семантика, дальние зависимости.

**На что влияют веса: они определяют, чьи Value попадут в представление текущего токена — и напрямую формируют следующее предсказание.**

Продакшен 2025-26: часть моделей заменяет часть слов на разреженное/линейное внимание — это снижает вычисления, но правило  $N \times N$  остаётся базой и ценой полного внимания.

21/47

Вы знаете слово «внимание» из каждой второй статьи про трансформеры; зафиксируем точную форму. Внимание — не линейная, а матричная операция: каждый токен контекста сверяется с каждым, и для контекста длины  $N$  карта весов имеет размер  $N$  на  $N$ . Из этой формы сразу следует главное экономическое свойство всей архитектуры: удвоение контекста учетверяет объём вычислений внимания. Когда дальше мы будем говорить о цене миллионных окон и о том, почему провайдеры так агрессивно кэшируют, — корень именно здесь. На слайде — упрощённая матрица семь на семь для предложения «Кот съел мышь, потому что он был голоден». Числа иллюстративные, реальные веса нормируются так, что сумма по строке равна единице. Качественно картина типична: в строке токена «он» наибольший вес лежит на токене «Кот», а не на «мышь» — местоимение мужского рода разрешается на одушевлённый субъект того же рода, стоящий в начале предложения. Это то, что мы в быту называем «модель поняла, к чему относится местоимение», — технически это статистическая связь, выученная на огромном корпусе. Два уточнения к картинке. Первое: распределение весов пересчитывается на каждом шаге генерации заново — это не «посчитали один раз и пользуемся». Второе: реальный механизм многослойный и многоголовый — в каждом слое параллельно работают десятки «голов», типично от 32 до 128, и каждая специализируется на своём типе связей: одна ловит грамматическое согласование, другая тематическую близость, третья дальние зависимости. Эти специализации никто не проектировал — они выросли из обучения, потому что помогают предсказывать следующий токен. И главное — на что конкретно влияет это распределение весов, а не просто «что-то там взвешивается». Вес каждой связи определяет, какая доля Value-вектора соответствующего токена попадёт в итоговое представление текущей позиции: если «он» отдаёт 0.7 веса «Кот», то именно содержимое «Кот» — его Value —

доминирует в обновлённом представлении токена «он». А представление текущего токена — это именно то, из чего на следующем шаге строится распределение вероятностей для предсказания. Другими словами: распределение весов внимания напрямую и механически определяет, каким будет следующий предсказанный токен — это не побочный эффект, а прямая причинная цепочка. Важная оговорка про 2025-2026:  $N \times N$  — точное описание одного слоя dense-внимания, и оно остаётся базой для понимания механизма. Но в реальных продакшен-моделях 2025-2026 не все слои dense: Qwen3-Next и Kimi Linear оставляют dense-внимание только в 25% слоёв, остальные — линейные ( $O(N)$ ); итоговая asymptotics всей модели уже не чисто квадратичная, но полностью линейной она тоже не становится, пока хотя бы часть слоёв остаётся dense — это компромисс, а не устранение проблемы.

## Внимание — распределение весов на весь контекст: три проекции Query / Key / Value

Q — про текущий шаг. K и V — про уже обработанный контекст.

**Query** — «что я сейчас ищу»

**Key** — «что я предлагаю»

**Value** — «что я отдам, если меня выбрали»

«Кот съел мышь, потому что он был голоден»

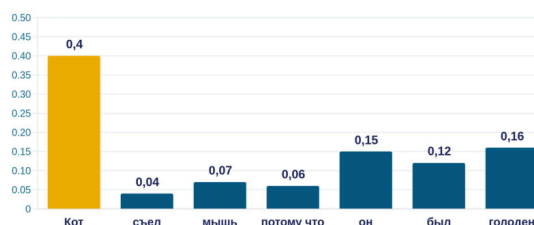
**Q («он»)** = «ищу: кто мог быть голоден»

**K («Кот»)** = «я — одушевлённый субъект»

**V («Кот»)** = содержимое, которое вольётся в представление «он»

Совпадение Q («он») и K («Кот») даёт высокий вес → V («Кот») определяет обновлённое представление токена «он».

Распределение весов — сумма = 1, лидер «Кот»



1. На вход — все токены контекста.

2. На выходе — распределение весов, сумма = 1.

3. Пересчитывается на каждом шаге генерации.

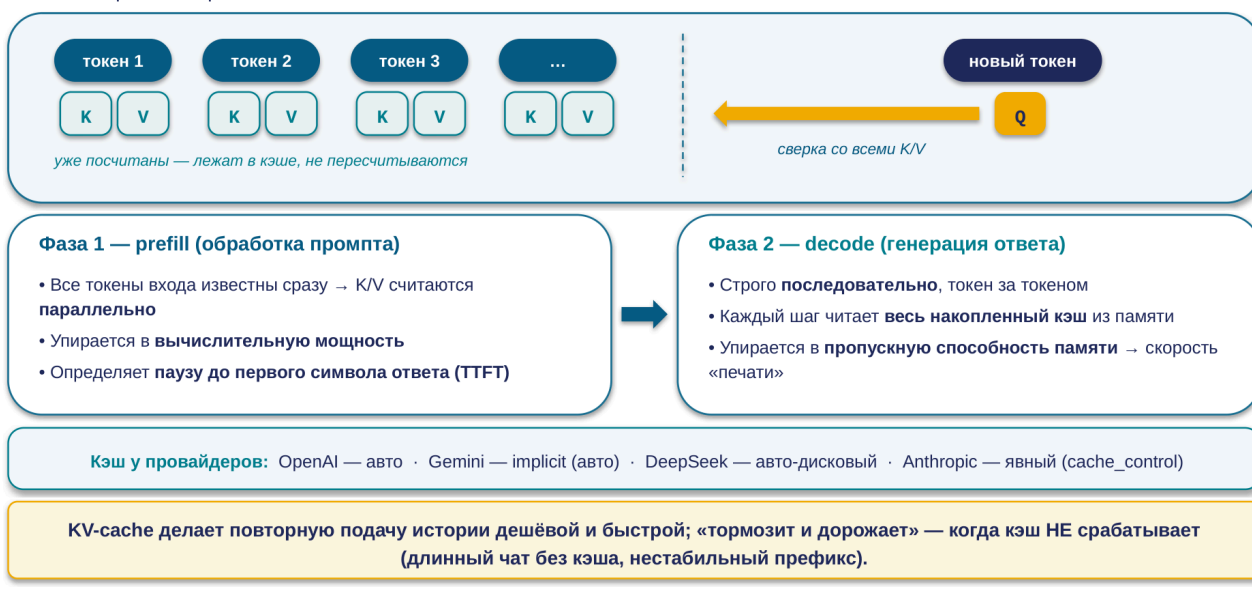
Метафора: фонарик в тёмной комнате — луч направлен на релевантные токены, яркость света = вес внимания.

22/47

Зафиксируем определение точно. Рабочая метафора — фонарик в тёмной комнате: все токены контекста присутствуют, но луч направлен на те, что релевантны текущему вопросу, и яркость по предметам — это веса. Формально на каждом шаге внимание возвращает распределение весов на все токены контекста, сумма равна единице, и это распределение пересчитывается на каждом шаге генерации заново. Теперь — один уровень глубже, на нашем же предложении, чтобы не потерять конкретику. Для каждого токена модель вычисляет три проекции его вектора: Query, Key и Value. Возьмём токен «он» из «Кот съел мышь, потому что он был голоден». Его Query можно грубо прочитать как «ищу: кто мог быть голоден» — запрос текущей позиции к остальному контексту. Токен «Кот» предлагает свой Key — нечто вроде «я — одушевлённый субъект», визитную карточку, по которой его находят чужие запросы. Когда Query «он» и Key «Кот» хорошо совпадают, вес между ними получается высоким — именно это мы видели в матрице на предыдущем слайде. А дальше в дело вступает Value «Кот» — содержимое, которое реально вливается в обновлённое представление токена «он». Итог позиции «он» — взвешенная сумма Value всего контекста, где Value «Кот» вносит наибольший вклад. Простыми словами: Query — это вопрос, который задаёт токен; Key — это то, чем токен отвечает на чужие вопросы; Value — это то, что токен реально передаёт, если его выбрали. Запомнить стоит одно структурное наблюдение: Query — про текущий шаг, а Key и Value — про уже обработанный контекст, который не меняется. Из этой асимметрии вырастает главная оптимизация всей индустрии вывода моделей — увидим её на следующем слайде, когда разберём, почему длинный чат тормозит.

## К и V кешируются — заново считается только Q

**KV-cache:** Key/Value обработанных токенов сохраняются в памяти ускорителя. На каждом шаге вычисляется только Q нового токена — против сохранённых K/V.



23/47

Вспомните наблюдение с прошлого слайда: Query — про текущий шаг, Key и Value — про уже обработанный контекст, который не меняется. Генерация авторегрессионна: токены выдаются по одному, и наивная реализация пересчитывала бы K и V всех токенов на каждом шаге заново — но они не меняются. Отсюда центральная оптимизация всего инференса больших моделей — KV-cache: Key- и Value-векторы вычисленных токенов сохраняются в памяти ускорителя, и на каждом шаге вычисляется только Q нового токена и его произведения с сохранёнными K/V. Из кэша вырастает асимметрия двух фаз, которую вы наблюдаете в каждом запросе. Prefill — обработка входного промпта: все его токены известны сразу, их K/V считаются параллельно одним большим матричным вычислением; фаза упирается в вычислительную мощность и определяет паузу до первого символа ответа. Decode — генерация: строго последовательная, и на каждом шаге нужно прочитать весь накопленный кэш из памяти; фаза упирается в пропускную способность памяти и определяет скорость в токенах в секунду. Поэтому длинный промпт прежде всего растягивает паузу до первого токена, а длинный контекст в целом замедляет каждый шаг генерации. Здесь важна точная формулировка вывода, потому что она легко искажается в народную мудрость «для новой задачи — новый чат». Пока KV-cache работает — то есть история совпадает с тем, что уже посчитано, — повторная подача истории дешёвая и быстрая: именно ради этого кэш существует. «Тормозит и дорожает» происходит не как свойство длинного чата вообще, а именно тогда, когда кэш не срабатывает — когда контекст меняется в начале, когда сессия истекла, когда провайдер вытеснил ваш кэш из памяти под чужую нагрузку. История подаётся в модель целиком при каждом обороте — модель без состояния, как мы фиксировали в Лекции 1, — но это не значит, что каждый оборот пересчитывается с нуля. Устройство кэша у провайдеров разное, и это

прямо влияет на то, что вам нужно делать руками. У OpenAI и у DeepSeek кэш срабатывает автоматически — без специальной настройки в запросе; у DeepSeek он к тому же дисковый, а не только в памяти ускорителя. У Google Gemini — `implicit caching`, тоже включён по умолчанию. А у Anthropic кэш явный: нужно расставить маркер ``cache_control`` в нужных местах промпта самостоятельно — иначе кэш не сработает, даже если содержимое совпадает. Практический вывод: если вы на Anthropic и не видите экономии — проверьте, стоит ли маркер там, где нужно; на остальных трёх — проверьте метрики `cache hit rate` в ответе API, потому что «автоматический» не значит «гарантированно срабатывающий». Масштаб эффекта: при контекстах в сотни тысяч токенов кэш одного пользователя занимает гигабайты памяти ускорителя — отсюда агрессивный батчинг провайдеров и премиальная цена длинных контекстов.

## Кэш промптов — ставка на повторное использование, а не скидка

### Три запроса подряд:

**Запрос 1:** префикс пишется в кэш — запись 1.25–2× ставки

**Запрос 2:** тот же префикс → cache hit — чтение 0.1× (новейшие — 0.025×)

**Запрос 3:** строка добавлена в начало → префикс не совпал → мимо кэша, полная цена

Кейс: 50 000 анализов документов в месяц

без кэша \$45 000

\$8 000 с кэшем · -82%



**Условие: точное совпадение префикса.** Один изменившийся токен инвалидирует кэш для всего, что после него.

системный промпт

✓ кэш

инструкции

✓ кэш

документы

✓ кэш

«Сегодня 2026-09-05 14:23» — динамический

!

вопрос

× мимо кэша

Динамический элемент в начале (timestamp) рушит кэш всех блоков после него — это и есть «запрос 3».

**Правило компоновки:** стабильное — в начало (промпт, инструкции, примеры, документы), переменное — в конец.

24/47

KV-cache живёт внутри одной генерации. Провайдеры сделали следующий шаг: если два разных запроса имеют одинаковый префикс — тот же системный промпт, инструкции, документы, — его K/V можно переиспользовать между запросами. Это кэширование промптов, и с 2025–2026 годов это главный рычаг оптимизации стоимости LLM-нагрузок. Проследим механику на трёх последовательных запросах — так виднее, чем на отдельных цифрах. Запрос первый: вы отправляете промпт впервые, кэша ещё нет — префикс записывается в кэш, и запись стоит дороже обычного входа, от 1.25 до 2 ставок в зависимости от времени жизни кэша. Запрос второй: вы отправляете тот же префикс — например, тот же системный промпт и тот же набор документов, только новый вопрос пользователя в конце — префикс совпадает побайтово, кэш срабатывает, и вы платите десятую часть базовой ставки входа, у новейших моделей — до одной сороковой. Запрос третий: кто-то добавил в начало промпта одну строку — например, текущую дату или обновлённую инструкцию — и теперь префикс больше не совпадает с тем, что в кэше, начиная с этой строки. Кэш мимо: вся нагрузка платит полную цену, как будто кэша никогда не было, а если у вас явная запись — ещё и надбавку за неё. Кэш не бесплатен — это ставка на повторное использование: окупается со второго-третьего попадания, а нагрузка из уникальных запросов без общего префикса только дорожает. Масштаб эффекта реален: кейс на 50 тысяч анализов документов в месяц — 45 тысяч долларов без кэша против 8 тысяч с кэшем, экономия 82 процента. Главное техническое условие видно на третьем запросе — точное совпадение префикса: кэш срабатывает, только если всё до контрольной точки совпадает байт-в-байт. Классический самострел — добавить в начало системного промпта что-то переменное, вроде текущей даты и времени: тогда ни один запрос никогда не попадёт в кэш. Отсюда правило компоновки: стабильное — в начало, переменное

— в конец. И минутный аудит для действующего конвейера: посмотрите поля учёта кэша в ответах API — если при стабильном системном промпте чтения из кэша нулевые, вы почти наверняка чините это перестановкой промпта за один вечер. Для многошаговых агентов кэш — не оптимизация, а условие рентабельности: к этому вернёмся в следующей лекции.

## Роль работает через вес во внимании — но не повышает фактическое качество

Рабочий пример: куда смотрит «он»

«Кот съел мышь, потому что он был голоден»

главный вес

Упрощение: агрегат сотен связей в десятках слоёв — модель воспроизводит корреляции употребления, не грамматический разбор.

Подумайте 30 секунд: куда уйдёт вес от «он» в «Программа упала, потому что он забыл обработать pull»?

Без роли

«Объясни GIL» → нейтральный, обобщённый ответ.

С ролью

«Ты — опытный Python-разработчик. Объясни GIL» — токены роли получают вес → смещают распределение следующих токенов: конкретнее, в экспертном регистре.

Zheng et al. (2024, EMNLP Findings; 2410 вопросов, 162 роли): персона/роль в промпте не повышает точность факта — эффект конкретной роли непредсказуем.

Отдельно от этого исследования — по наблюдениям курса: роль ощутимо меняет тон, стиль и отбор содержания ответа.

Роль — инструмент управления стилем и фокусом, а не «усилитель ума». Нужен ответ по вашим данным — дайте данные, а не третье прилагательное к слову «эксперт».

25/47

Разминка: «Кот съел мышь, потому что он был голоден». Дойдя до токена «он», модель должна разрешить, к кому он относится, — и в карте внимания вес распределён в пользу «Кот». Три стрелки — упрощение: агрегат сотен связей в десятках слоёв; модель не делает грамматический разбор, а воспроизводит корреляции употребления. Проверьте интуицию: «Программа упала, потому что он забыл обработать pull». На большинстве моделей вес уйдёт к ближайшему подходящему одушевлённому существительному по смыслу контекста — если ваша интуиция дала похожий ответ, вы уже думаете в терминах корпусной статистики. Теперь — то, ради чего вы это уже знаете. Сравним два одинаковых по сути промпта. «Объясни GIL» без роли — модель отвечает нейтрально, обобщённо. «Ты — опытный Python-разработчик. Объясни GIL» — вы наверняка замечали, что такой промпт работает иначе. Механизм: токены «опытный», «Python-разработчик» получают вес в распределении внимания при генерации каждого токена ответа, и выбор смещается в согласованную с ними сторону — ответ становится конкретнее и увереннее, в экспертном регистре. Роль — не просьба «поверь мне», а входной сигнал, физически участвующий во взвешивании. И граница эффекта, которую важно знать до того, как вы начнёте злоупотреблять ролями в промптах: Zheng et al. (2024, EMNLP Findings) проверили 2410 вопросов и 162 роли — персона/роль в промпте не повышает точность факта, эффект конкретной роли непредсказуем. Отдельно от этого исследования: по наблюдениям курса роль ощутимо меняет тон, стиль и отбор содержания ответа — это отдельный, не тождественный факту эффект. Роль смещает распределение, не добавляет знаний. Если модель не знает ответа, роль «эксперт» не заставит её узнать — она заставит её увереннее звучать, не будучи более правой. Практический вывод: роль — рабочий инструмент управления стилем и фокусом ответа, полезный и легитимный, но не путайте его с

повышением точности. Нужен точный ответ по вашим данным — дайте модели данные, а не третье прилагательное к слову «эксперт».

## Фронтир-стандарт 2026 — окно до 1 млн токенов. Но заявленное ≠ полезное

GPT-3.5 (2022) — 4 тыс.



Claude 3.5 (2024) — 200 тыс.



Фронтир-стандарт (2026) — до 1 млн · Fable 5, GPT-5.6, Gemini 3.1 Pro и др.



Gemini 3.5 Pro (2026) — 2 млн · единичная модель, не стандарт



ширина — логарифмическая шкала

Llama 4 Scout: «10 млн»

заявлено; ни один опубликованный бенчмарк не подтверждает качество вблизи предела

YandexGPT 5 Pro: 32 тыс.

на полтора-два порядка меньше флагманов — для длинных документов это определяющее ограничение

Просто «растянуть» окно нельзя: позиция токена закодирована геометрией, обученной на конкретных длинах, — расширение (RoPE / YaRN) — отдельная инженерная работа.

10M заявка ≈ 32 TB KV-cache — физический потолок.

Платите за то, что кладёте в окно, а не за то, что окно вмещает: 900 тыс. токенов входа по \$10/млн ≈ \$9 за один вызов.

Что делать: выбирайте модель по эффективному окну задачи (бенчмарки без лексических подсказок), не по маркетинговому максимуму.

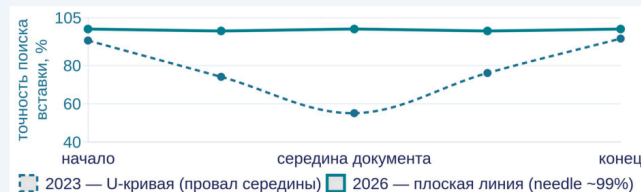
26/47

Контекстное окно — максимальное число токенов на один запрос — за четыре года выросло на три порядка: 4 тысячи у GPT-3.5 в момент запуска ChatGPT, 200 тысяч у Claude 3.5 в 2024-м, и фронт-стандарт 2026 года — до одного миллиона: Fable 5, GPT-5.6, Gemini 3.1 Pro и другие держат этот уровень, причём у части моделей полное окно входит в стандартную цену без наценки за длину. Два миллиона у Gemini 3.5 Pro — не стандарт, а полюс: пока единственная модель на этом уровне, а не типичный флагман; моделей с окном до миллиона на рынке уже больше десятка. Два отрезвляющих полюса вокруг стандарта. Сверху — маркетинг: Llama 4 Scout заявляет десять миллионов токенов, но ни один опубликованный бенчмарк не подтверждает сохранение качества вблизи этого предела; «заявленное окно» и «полезное окно» — разные величины, и разрыв между ними мы измерим через слайд. Снизу — контраст: YandexGPT 5 Pro работает с окном 32 тысячи — для задач с длинными документами это не нюанс, а определяющее ограничение выбора. Почему окно конечно и его нельзя «просто увеличить»? Тут две разные причины, и их стоит разделять. Первая — вычисления: даже там, где часть слоёв разреженная или линейная, цена всё равно растёт быстрее линейной на оставшихся dense-слоях. Вторая — память: KV-cache растёт линейно с числом токенов независимо от того, dense внимание или sparse, — разреженность решает, к какой части кэша обращаться, а не сколько его вообще нужно хранить; это отдельно решают MLA, GQA, квантизация, sliding window. На практике это не абстракция: при заявленных десяти миллионах токенов у Llama 4 Scout один только KV-cache требует порядка тридцати двух терабайт памяти — это физический, а не алгоритмический потолок. И третья причина, отдельная от compute/memory: позиция токена закодирована в геометрии модели способом, обученным на конкретных длинах, и наивное растяжение ломает механизм; методы расширения — RoPE, YaRN — разобраны в

методичке для самостоятельного чтения: глава, часть 2 — раздел о позиционном кодировании. И арифметика денег, которую полезно проделать один раз. Полное окно — это токены, которые вы оплачиваете как вход при каждом запросе: запрос к премиальной модели по \$10 за миллион входных токенов, заполненный на 900 тысяч, стоит около \$9 — за один вызов; десять оборотов диалога — под сто долларов без кэширования. Вопрос «сколько контекста нужно этой задаче» экономически важнее вопроса «сколько модель может принять».

## Поиск дословной вставки решён. Понимание длинного контекста — нет

Забывание победили ✓ — найти дословную вставку (needle-in-a-haystack)



Работу со смыслом — нет (NoLiMa, 2025: бенчмарк убрал буквальное совпадение слов)



Без лексических совпадений качество падает ниже 50% от базового уже на 32K — у 11 из 13 моделей.

32 тыс. токенов ≈ 3% заявленного окна флагамена; база — точность той же модели на коротком контексте.

1M окна ≠ 1M рассуждения. Окно — сколько модель может прочитать; полезная длина — на скольких токенах она ещё связывает факты.

27/47

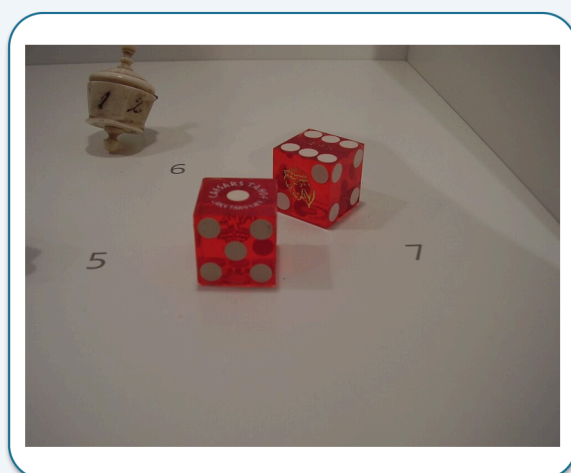
Вы, вероятно, знаете классический результат 2023 года «Lost in the Middle»: факт из середины длинного контекста извлекается хуже, чем с краёв, — U-образная кривая точности. Это знание требует обновления 2026 года — в обе стороны. Хорошая новость: буквальный поиск «иголки в стоге» — найти дословно вставленную фразу — флагманами практически решён: до 99% на полном миллионном окне. Если задача — найти, где в договоре упоминается сумма, большое окно работает почти как обещано. Плохая новость пришла от бенчмарка NoLiMa, который убрал главный костыль таких тестов — буквальное лексическое совпадение между вопросом и спрятанным фрагментом. Когда искомое нужно узнать по смыслу, а не по совпадающим словам, картина обрушивается: одиннадцать из тринадцати протестированных моделей падают ниже половины их же собственной точности на коротком контексте — обратите внимание на базу: сравнение модели с самой собой, а не абсолютный порог. И происходит это уже на 32 тысячах токенов — не на миллионе, а на трёх процентах заявленного окна флагамена. U-кривая не исчезла — она спряталась за высокими цифрами тестов, которые измеряют поиск, а не рассуждение. Формула для запоминания: миллион окна не равен миллиону рассуждения. Окно — это сколько модель может прочитать; полезная длина — на скольких токенах она ещё способна связывать факты без буквальных подсказок, и вторая величина во много раз меньше первой. Следствия: критические инструкции — в начало или конец, не в середину; «вывалить всю базу знаний в окно» проигрывает хорошему поиску с пятью-десятью целевыми фрагментами — про это следующая лекция; и при выборе модели для длинных документов спрашивайте не «какое окно», а «какие результаты на бенчмарках без лексического совпадения» — и тестируйте на рабочей длине вашей задачи: полчаса стресс-теста на реальных документах говорят больше любого лидерборда.

# Раздел 4

## Сэмплинг и генерация

*«Как из распределения вероятностей рождается один токен — и какими ручками этот выбор управляется»*

4 разбора · 2 провала



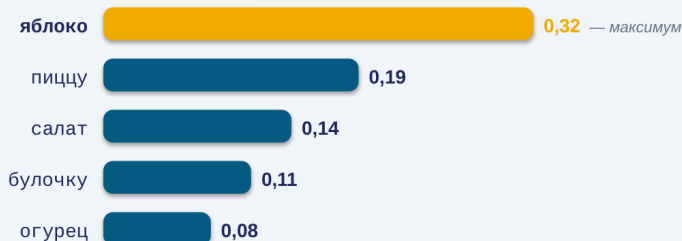
28/47

Три стадии конвейера позади: текст разрезан на токены, токены получили векторы, внимание решило, на что опереться в контексте. Итог этих трёх стадий — распределение вероятностей на весь словарь модели: для каждого из примерно двухсот тысяч токенов — вероятность оказаться следующим. Четвёртая стадия — сэмплинг: правило выбора одного токена из этого распределения. Всё «творчество» и вся «случайность» больших моделей живут в этом правиле, и оно — единственная часть конвейера, которой вы управляете напрямую, ручками API. Порядок движения по разделу. Сначала зафиксируем само распределение — что оно и есть «настоящий» выход модели, а всё остальное — политики поверх него. Затем точная механика температуры и её соседей — с живым сравнением поведения на нуле и на полутора. Дальше — проверка самого коварного из шести утверждений: действительно ли ноль температуры даёт одинаковые ответы; ответ вас, скорее всего, удивит и он важен для каждого, кто пишет тесты на LLM. После этого — как классический набор из четырёх ручек API вырос до шести, гарантированный формат ответа через структурированный вывод, авторегрессионный цикл, собирающий всё в единое целое, и токены рассуждения — невидимая, но платная статья вашего счёта. Завершим раздел свежим взглядом на выбор между локальной моделью и облаком.

## На каждом шаге — распределение вероятностей на все токены словаря

Контекст: «Сегодня я съел ...»

*P(следующий токен):*



Сэмплинг → один токен

правило выбора одного токена из распределения — единственная ручка в ваших руках



[ яблоко ]

выбран один — остальные кандидаты исчезли из ответа

*Остальные ~200 000 токенов словаря — каждый < 0.05. Сумма всех вероятностей = 1. Числа иллюстративные.*

**Распределение — «настоящий» выход модели. Уверенный ответ и галлюцинация до сэмплинга существовали одновременно — как вероятностная масса; выбор сделала политика.**

29/47

Итог трёх стадий конвейера — на выходе модели лежит распределение вероятностей на весь словарь: для каждого из примерно двухсот тысяч токенов — вероятность оказаться следующим; сумма равна единице. На запросе «Сегодня я съел» распределение выглядит примерно так: «яблоко» — 0.32, «пиццу» — 0.19, «салат» — 0.14, дальше длинный хвост; числа иллюстративные, картина устойчива. Распределение не равномерное — у модели есть статистические предпочтения, — но и не точечное: правдоподобных кандидатов несколько. Сэмплинг — правило выбора одного токена из этого распределения. Всё «творчество» и вся «случайность» больших моделей живут в этом правиле, и оно — единственная часть конвейера, которой вы управляете напрямую, ручками API. Стоит один раз осознать, насколько это распределение — «настоящий» выход модели, а всё остальное — политики поверх него. Уверенный ответ и уклончивый, точный факт и галлюцинация, «Париж» и «возможно, вы имеете в виду...» — до сэмплинга всё это существовало одновременно, как вероятностная масса на разных продолжениях; выбор сделала политика, а не «мнение» модели. Практическое следствие для тех, кто строит конвейеры: часть провайдеров отдаёт срез этой информации в API — лог-вероятности топ-кандидатов на каждом шаге. Это полезный диагностический канал: широкое, размазанное распределение на ключевом токене ответа — честный сигнал неуверенности модели, который сам текст ответа может никак не выдавать. Для классификации это дешёвый способ получить меру уверенности без второго запроса «а насколько ты уверена?» — второй запрос, кстати, измеряет не уверенность модели, а её натренированность отвечать на такие вопросы.

## Температура — делитель логитов: меняет остроту выбора, не знания

$T = 0 \Rightarrow \text{выбор} = \text{argmax } P(\text{токен})$      $T > 0 \Rightarrow \text{сэмплинг из } P^{(1/T)}\text{-образного распределения}$

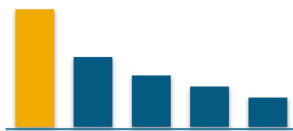
логиты делятся на  $T$  до softmax:  $T < 1$  заостряет распределение,  $T > 1$  выравнивает

$T \rightarrow 0$  (argmax)



Выбор самого вероятного. Почти одинаковые ответы — «почти» разберём на следующем слайде.

$T = 1$  (стандарт)



Сэмплирование пропорционально вероятностям модели.

$T = 1.5$  (сглаживание)



Редкие токены получают реальные шансы — до бессвязности.

Почему  $T$  влияет, хотя порядок кандидатов не меняется: выбор — не «взять топ-1», а случайная выборка пропорционально вероятностям;  $T$  перекраивает сами вероятности.

**top-p** — отрез хвоста по вероятностной массе · **top-k** — по числу кандидатов ·  $T$  под задачу: 0–0.3 код и классификация, 0.7+ генерация.

**Живой прогон:** один запрос — 10 раз при  $T=0$  и 10 раз при  $T=1.5$ .

30/47

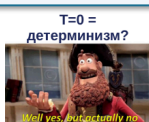
Эти ручки вы знаете — уточним механику до точной формулы. При  $T=0$  выбор — это  $\text{argmax } P(\text{токен})$ : детерминированно берём токен с максимальной вероятностью, без всякого сэмплирования. При  $T>0$  модель сэмплирует из распределения, преобразованного по правилу  $P$  в степени  $1/T$  — технически это означает, что сырые логиты делятся на  $T$  перед тем, как пройти через softmax и превратиться в вероятности. Здесь стоит остановиться на вопросе, который часто вызывает путаницу: порядок токенов по вероятности не меняется от температуры — самый вероятный токен остаётся самым вероятным при любой  $T$ . Тогда почему температура вообще на что-то влияет, если ранжирование одно и то же? Ответ в том, что выбор токена — это не «взять топ-1 из списка», а случайная выборка, где вероятность выбора каждого токена пропорциональна его вероятности в распределении. А температура меняет сами эти вероятности, не их порядок: при  $T$  меньше единицы распределение заостряется — лидер получает ещё больше массы вероятности, аутсайдеры теряют шансы почти до нуля; при  $T$  больше единицы распределение выравнивается — разрыв между лидером и остальными токенами сокращается, и у токенов, которые раньше были почти невозможны, появляется реальный шанс быть выбранными. Именно поэтому  $T=0$  — предельный случай: заострение доведено до предела, и вероятность лидера становится равна единице —  $\text{argmax}$ . Второй эшелон: top-p отрезает хвост по вероятностной массе — сэмплируем из минимального набора самых вероятных токенов с суммарной вероятностью не меньше  $p$ ; top-k отрезает по числу кандидатов. Практика не изменилась: основная ручка — температура, эти две — тонкая настройка. Теперь живой прогон: один и тот же запрос — «придумай название для сервиса заметок» — десять раз при нуле и десять раз при полутора. Первый ряд даёт почти идентичные ответы — «Заметкин», «Заметкин», «Заметкин», изредка меняется разве что окончание; слово «почти»

здесь несёт больше смысла, чем кажется, и ему посвящён следующий слайд. Второй ряд — разброс от удачных находок до бессвязицы: «МыслеПоток», «конспект-туман на рассвете», «блокнот дыхания чисел» (примеры иллюстративные — свой разнбой вы увидите за минуту). Ценность этого эксперимента, если повторите его сами, не в подтверждении теории, а в калибровке руки: вы увидите, как выглядит хвост распределения вашей конкретной задачи и на какой температуре он начинает протекать в ответы. И обратное упражнение: если ваши промпты в проде работают на температуре по умолчанию — выясните, какая она у вашего провайдера; значения по умолчанию у всех разные и меняются, а осознанный выбор здесь стоит одну строку кода.

## T=0 не даёт детерминизма: 80 уникальных ответов из 1000

# 80 / 1000

уникальных вариантов ответа на идентичный запрос при T=0 — стандартный vLLM (открытый инференс-сервер; Thinking Machines Lab, сентябрь 2025)



**Решение существует:** batch-инвариантные ядра — 1000/1000 побитово идентичны

Причина — не «floating-point вообще», а отсутствие batch-инвариантности ядер:

1. Сервер группирует одновременные запросы разных пользователей в батчи
2. Размер батча зависит от чужой нагрузки в эту миллисекунду
3. Разный размер батча → разный порядок суммирования → младшие разряды другие
4. Два близких кандидата в argmax → младший разряд решает выбор токена → авторегрессия разносит расхождение по всему ответу

**Цена:** ~35% пропускной способности → провайдеры не включают; seed у OpenAI — официально «mostly deterministic», в основном детерминировано

Получить гарантированно детерминированный ответ от облачной LLM сегодня нельзя — стройте процессы с учётом этого. Следствие: тесты — не на побитовом сравнении, а семантически или по структуре.

31/47

Это утверждение — самое коварное из шести, потому что оно почти правда, и на нём строят тесты и конвейеры. Вы ставите ноль температуры, ожидая: одинаковый запрос — одинаковый ответ. Проверка на реальной инфраструктуре: стандартный vLLM, тысяча запусков идентичного запроса — восемьдесят уникальных вариантов ответа. Ноль действительно делает выбор argmax-детерминированным — но само распределение, из которого берётся argmax, оказывается чуть разным от запуска к запуску. Причина глубже, чем расхожее «floating-point на GPU». Главный виновник — отсутствие batch-инвариантности вычислительных ядер. Сервер провайдера динамически группирует одновременные запросы разных пользователей в батчи; размер батча зависит от нагрузки в конкретную миллисекунду; а многие ядра при разном размере батча используют разный порядок суммирования. Сложение чисел с плавающей точкой неассоциативно — порядок меняет младшие разряды, — и когда два кандидата в argmax близки, младший разряд решает выбор токена; дальше авторегрессия разносит расхождение по всему ответу. Ваш «детерминированный» запрос недетерминирован, потому что вы делите сервер с другими пользователями, и их трафик меняет размер вашего батча. Самое поучительное: проблема решается — и решение отвергнуто экономикой. Те же авторы написали batch-инвариантные ядра: тысяча из тысячи запусков побитово идентичны. Цена — около 35 процентов пропускной способности, поэтому провайдеры этот режим по умолчанию не включают; отсюда и статус параметра seed у OpenAI — официально «в основном детерминировано», без гарантий. Главный вывод формулируем так, и порядок здесь важен: получить гарантированно детерминированный ответ от облачной LLM сегодня нельзя — это не временный баг, а следствие экономического выбора провайдеров, и проектировать процессы нужно с учётом этого факта, а не в надежде, что «в

следующей версии починят». Отсюда, как следствие, а не как главный тезис: не стройте тесты на побитовом сравнении ответов — сравнивайте семантически или по структуре; воспроизводимость фиксируйте датасетом и метрикой. А если строгий детерминизм действительно нужен — аудит, регуляторика, — это отдельное требование к инфраструктуре с ценником в треть пропускной способности, заложенное в бюджет с самого начала.

### Ручки API: к случайности и длине добавилась глубина рассуждения

| Параметр    | Диапазон     | На что влияет                                         | Типичное значение                 |
|-------------|--------------|-------------------------------------------------------|-----------------------------------|
| temperature | 0–2          | Детерминизм ↔ хаос выбора токена                      | 0 классификация; 0.7–1.2 текст    |
| top_p       | 0.1–1        | Ширина хвоста кандидатов при сэмплинге                | 0.9–0.95                          |
| max_tokens  | целое        | Жёсткий обрыв генерации — может прервать на середине  | под задачу, с запасом на JSON/код |
| effort      | none → xhigh | Глубина внутреннего рассуждения — и его цена          | medium у большинства провайдеров  |
| verbosity   | low → high   | Длина видимого ответа, независимо от глубины мышления | medium                            |

effort / reasoning\_effort — новая ось 2026: OpenAI — шкала effort; Anthropic — адаптивное мышление; Gemini — thinking budget.

Что делать: начинайте настройку с temperature и effort — две главные ручки; top\_p / top\_k / verbosity — тонкая настройка поверх.

32/47

Классический набор — температура, top\_p, max\_tokens — вы знаете; таблица параметров на слайде фиксирует диапазон и типичное значение для каждого: ноль для классификации, 0.2–0.3 для кода, около 0.7 для объяснений, 0.9–1.2 для творческих текстов. Новость 2026 года — у reasoning-моделей выросли две новые ручки, и они управляют другой осью. Effort, или reasoning\_effort, — глубина внутреннего рассуждения: у OpenAI шкала от «none» до «xhigh», у Anthropic — параметр effort при адаптивном мышлении, у Gemini — thinking budget, где ноль выключает рассуждение, а минус единица отдаёт выбор модели. Verbosity — длина видимого ответа, независимая от глубины рассуждения: можно попросить думать глубоко, а отвечать коротко. Старая пара «температура управляет случайностью, max\_tokens — длиной» дополнилась парой «effort управляет мышлением, verbosity — многословием». Живой пример эволюции API — полезный как прививка от заучивания параметров: Anthropic в 2026 году сломала обратную совместимость управления мышлением — ручной budget\_tokens на новых моделях возвращает ошибку 400; вместо него — адаптивное мышление, где глубину решает модель. Деталь на стыке с кэшированием: смена конфигурации мышления между запросами инвалидирует кэш промпта — она входит в кэшируемый префикс. И смежное следствие: не переносите привычки между провайдерами и поколениями. Ручки с одинаковыми именами ведут себя по-разному, а рекомендации меняют знак: OpenAI прямо не рекомендует давать reasoning-моделям промпты в стиле «давай подумаем шаг за шагом» — модель рассуждает сама, и ручная накрутка цепочки только дублирует работу и расход. Приём, годами бывший лучшей практикой, стал антипаттерном. При смене модели перечитывайте страницу параметров так же внимательно, как список ломающих изменений библиотеки при мажорном обновлении.

## Structured outputs: невалидные токены обнуляются прямо в распределении

**Constrained decoding:** JSON-схема компилируется в **конечный автомат над токенами**. При потоковой генерации автомат отслеживает состояние префикса и на каждом шаге **МАСКИРУЕТ** (обнуляет вероятность) токены, ведущие к невалидному продолжению.



Просьба «ответь строго в JSON» → ~80% валидных

**Strict mode** → **100%** — выход валиден по построению, а не «проверен постфактум»

### Ограничения — свойства компиляции

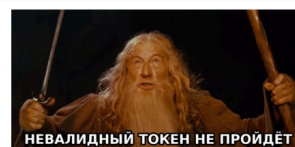
Рекурсия через \$ref — нет (дерево → плоский список с parent\_id)

Глубина вложенности ≤ 5

Первый запрос с новой схемой платит за компиляцию грамматики — до 10 с

Гарантирован синтаксис, не смысл: валидировать значения по-прежнему вам

**Вопрос залу:** почему именно 100%, а не 99.9?



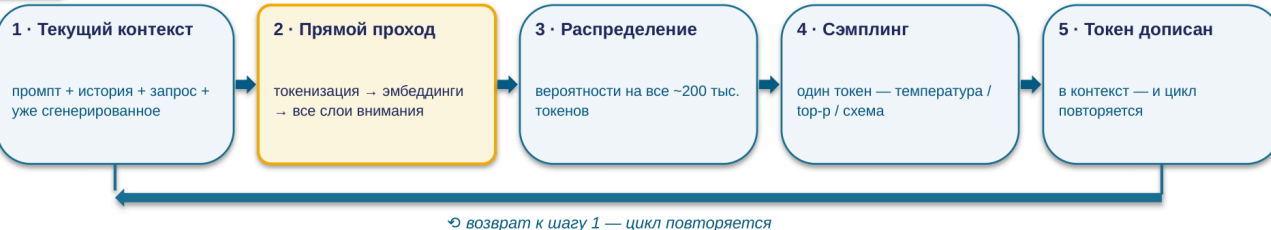
33/47

Все, кто просил модель «ответь строго в JSON», знают цену слова «строго»: обычная просьба даёт валидный JSON примерно в восьмидесяти процентах случаев, и оставшиеся двадцать ломают конвейер. Structured outputs решают задачу не уговорами, а механикой: заданная схема компилируется в конечный автомат над токенами; при потоковой генерации автомат отслеживает состояние уже сгенерированного префикса и на каждом шаге маскирует — обнуляет вероятность — токены, ведущие к невалидному продолжению, прямо в том распределении, которое мы разобрали в начале раздела. Поэтому выход валиден по построению, а не «проверен постфактум». Вопрос залу, тридцать секунд: почему заявлено ровно сто процентов, а не 99.9? Ответ: потому что гарантия встроена в сам сэмплинг — модель физически не может выбрать токен, нарушающий схему; это не проверка постфактум с повторным запросом, а фильтр в момент выбора. У OpenAI это strict mode, у Anthropic — нативный output\_format, у Gemini — response\_schema. Понимание механизма сразу объясняет ограничения — они не капризы API, а свойства компиляции грамматики. Схему нужно скомпилировать в автомат над токенами: рекурсия через ссылки не поддерживается — настоящее дерево неограниченной глубины не выражается конечной грамматикой, вложенные структуры моделируются плоским списком с полем родителя; глубина вложенности ограничена пятью уровнями; первый запрос с новой схемой платит за компиляцию — обычно до десяти секунд. Поэтому не генерируйте схемы динамически на каждый запрос — фиксируйте и версионите их как код. И последняя граница: гарантирован синтаксис, не осмысленность полей — валидировать значения по-прежнему нужно вам. Проектная рекомендация: начинайте со схемы проще, чем хочется, — плоские структуры работают надёжно и дёшево, а каждая степень вложенности приближает к границам компилятора. Схема — интерфейсный контракт с

недетерминированным исполнителем; чем контракт короче, тем меньше способов его нарушить.

## Предсказали token → дописали в контекст → предсказываем следующий

вход



## Как цикл останавливается

Специальный **стоп-токен** — модель сама решает «ответ закончен» — или **max\_tokens**: обрыв мгновенный, хоть на середине JSON-поля.

## Как цикл ломается

Вырожденный цикл повторов: модель «зацикливается» на одном токене или фразе и генерирует полотно повторов вместо остановки.



**Практика:** `repetition_penalty` / `frequency_penalty` снижают вероятность буквальных повторов; `max_tokens` — страховка от бесконечного цикла.

*Каждый шаг — без состояния: вся «память» живёт в контексте, который подаётся целиком (KV-cache делает подачу дешёвой, не отменяя её логически).*

34/47

Соберём конвейер в цикл — это ядро всей картины. Авторегрессионная генерация: текущий контекст — системный промт, история, запрос и уже сгенерированная часть ответа — проходит прямым проходом через токенизацию, эмбединги и все слои внимания; на выходе — распределение вероятностей следующего токена; сэмплинг выбирает один токен; токен дописывается в контекст; цикл повторяется до специального стоп-токена или лимита `max_tokens`. Каждый шаг — без состояния: вся «память» живёт в контексте, который подаётся целиком, — с поправкой на KV-cache, который делает повторную подачу дешёвой, не отменяя её логически. Ответ печатается на глазах не для красоты — это физический темп: токен за проход. Цикл может остановиться правильно — а может сломаться. Правильная остановка — специальный стоп-токен или лимит `max_tokens`; обрыв по лимиту мгновенный, хоть на середине JSON-поля, и постобработка обязана этот случай ловить. Поломка — вырожденный цикл повторов: модель зацкливается на одном токене или короткой фразе и генерирует полотно повторов вместо содержательного продолжения — «горшочек, не вари!» из сказки, только горшочек — это модель, а каша — токены. [Место для личной истории лектора: если у вас был реальный случай, когда модель ушла в такой цикл на проде или в тесте, — расскажите его здесь своими словами.] Механизм тот же цикл: если распределение на каком-то шаге сильно смещено в сторону уже сгенерированного текста (например, из-за низкой температуры или структуры промпта), модель раз за разом выбирает похожий токен, и авторегрессия закрепляет паттерн. Практика: `repetition_penalty` и `frequency_penalty` снижают вероятность буквальных повторов уже сгенерированных токенов; `max_tokens` в любом случае остаётся страховкой — даже если модель зацклилась, генерация не продолжится бесконечно. Сквозная трассировка — соберём весь конвейер на одном примере. Вы отправляете

«Столица Франции —». Токенизация: текст режется на три-четыре токена по таблице, зафиксированной до обучения. Эмбединги: каждый идентификатор забирает свой вектор из входной таблицы. Внимание: на позиции после тире Query «спрашивает», Key и Value слов «столица» и «Франции» отвечают весами — и они, кстати, уже лежат в кэше и не пересчитываются. Выход: распределение, где «Париж» получил, скажем, 0.93. Сэмплинг: при нуле температуры берём максимум — «Париж»; цикл делает ещё один оборот и выбирает стоп-токен. Если бы был включён режим рассуждения — перед «Парижем» цикл намотал бы черновые токены; если бы стояла JSON-схема — распределение на каждом шаге фильтровалось бы грамматикой. Один запрос — все стадии, все ручки. Терминологическая норма курса: «авторегрессионный», не «авторегрессивный».

## Reasoning-токены не видны — но тарифицируются как output



35/47

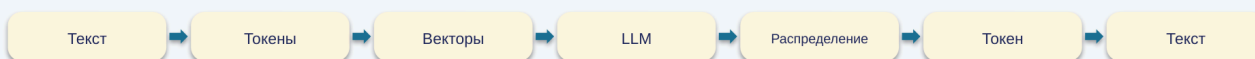
Reasoning-модели — o-серия OpenAI, расширенное мышление Claude, Deep Think у Gemini — не меняют цикл, который мы только что собрали. Они делают другое: перед видимым ответом модель генерирует тем же авторегрессионным циклом токены рассуждения — черновик «для себя», который в ответ не попадает. Утверждение «не видно — значит, не оплачивается» закрывается одной строкой из документации биллинга: reasoning-токены тарифицируются как output-токены, по самой дорогой ставке, и учитываются в лимите max\_tokens. Масштаб стоит прочувствовать в числах. Объём невидимого рассуждения раздувается в три-десять раз относительно видимого ответа, без естественного потолка. В типичной агентной нагрузке o3-pro на максимальной глубине рассуждения обходился порядка 280 долларов в месяц — в три и шесть десятых раза дороже o3 и в восемнадцать раз дороже o4-mini, — при том что видимые ответы всех трёх сопоставимы по длине; разницу делает именно объём рассуждения. Планируя бюджет, закладывайте невидимую часть в два-пять раз больше видимого ответа — и перепроверяйте по полю usage в ответе API, где эти токены видны как строка расхода. Ещё две границы. Первая: то, что вы видите как «ход рассуждений» в интерфейсе, — пересказ: провайдеры по умолчанию отдают summarized thinking, отдельный сгенерированный текст, а не сырые токены; строить на нём аудит решений нельзя. Вторая: управление сместилось от ручных бюджетов к адаптивности — модель сама решает, сколько думать. Это удобно — и это же означает, что стоимость запроса стала менее предсказуемой: на массовой обработке прижимайте effort вниз явно, а «думающий» режим включайте там, где сложность это оправдывает. Дефолтная глубина часто избыточна: если вы не измеряли зависимость качества от effort на своей задаче — с высокой вероятностью переплачиваете вдвое.

# Раздел 5

## Виды и размеры моделей

«На чём запускаются модели разных размеров — и что каждый класс умеет»

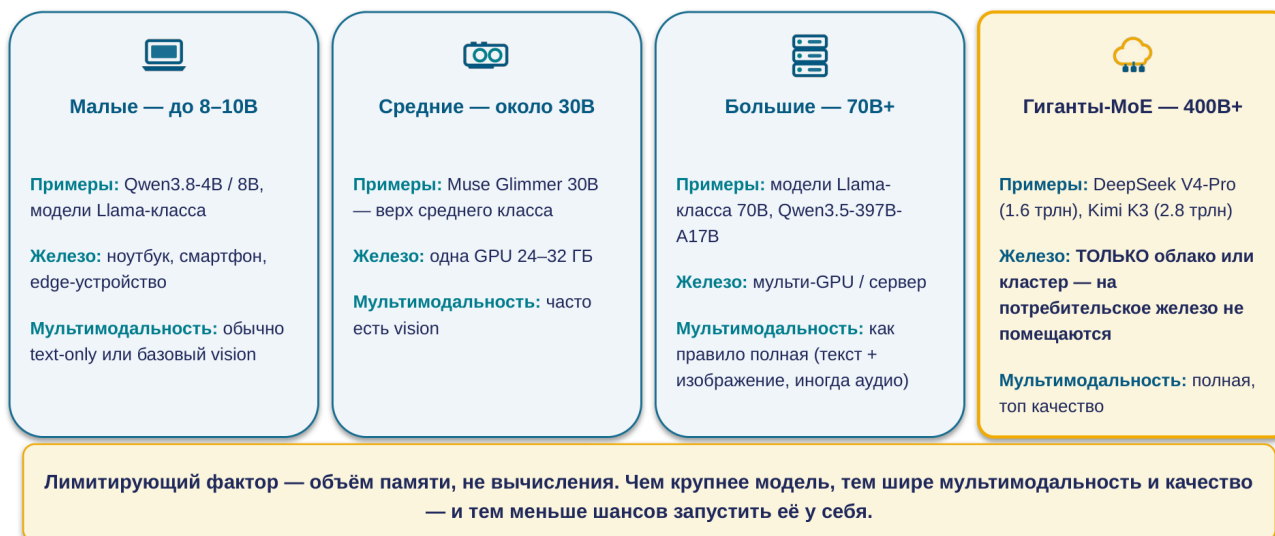
4 разбора



36/47

Конвейер из четырёх разделов — токенизация, эмбединги, внимание, сэмплинг — один и тот же независимо от того, какая по размеру модель его исполняет. Но размер модели определяет, где и как вы её запускаете, и это отдельный практический вопрос, который стоит разобрать явно. В этом разделе — четыре разбора. Сначала классификация по размеру: маленькие модели, которые помещаются на ноутбук или смартфон, средние на одну игровую видеокарту, большие на сервер с несколькими картами, и модели-гиганты, которые физически существуют только в облаке или на кластере, — с примерами конкретных моделей на сентябрь 2026 года и тем, что каждый класс умеет по мультимодальности. Затем — местозапуска отдельно от размера: тот же вопрос под другим углом, критерий выбора между локальным развёртыванием и облачным API. После этого — карта местности: ландшафт конкретных моделей на сентябрь 2026 года, передний край и открытые веса, цены. И завершит раздел жёсткий разговор о том, почему цифрам бенчмарков нельзя верить на слово при выборе модели. Это последний содержательный раздел перед финалом лекции — дальше мы соберём весь конвейер и все разобранные механизмы в одну картину.

## Четыре класса моделей по размеру — от ноутбука до кластера

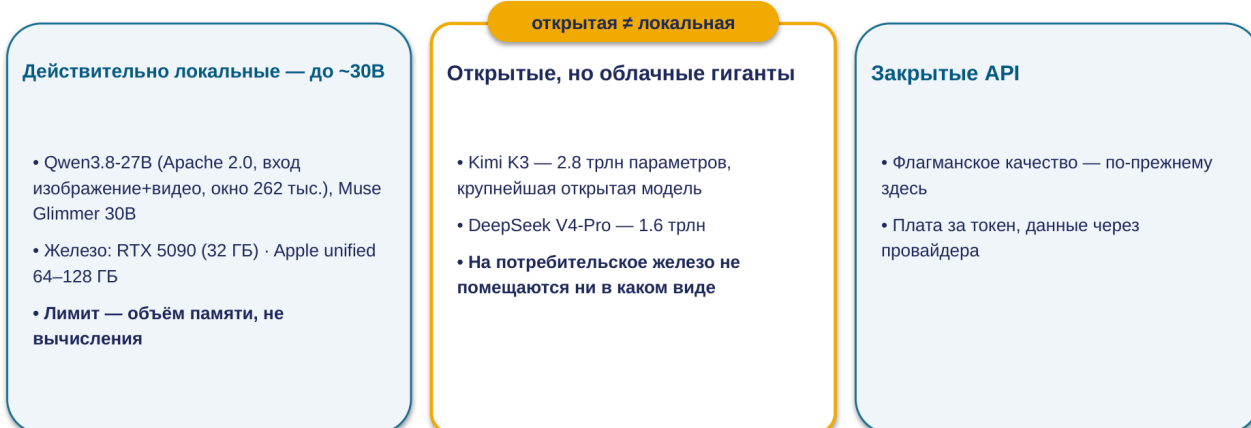


37/47

Прежде чем говорить, где запускать модель — локально или в облаке, — разберём другую ось: сам размер модели. Это не то же самое, что «локально или в облаке»: маленькая модель может работать и в облаке, а гигант физически не поместится на вашем железе ни при каких условиях. Классифицируем по числу параметров — от миллиардов до триллионов. Малые модели — до 8–10 миллиардов параметров — это класс, который реально работает на ноутбуке или смартфоне: Qwen3.8 в вариантах 4B и 8B, модели Llama-класса. По мультимодальности это обычно текстовые модели или модели с базовой поддержкой изображений — обучающая выборка и архитектура таких размеров ограничивают широту навыков. Средние модели — в районе 30 миллиардов параметров — это Muse Glimmer 30B, верхняя граница этого класса: такая модель умещается на одну игровую видеокарту с 24–32 гигабайтами памяти, и здесь уже часто встречается полноценная поддержка изображений на входе. Большие модели — от 70 миллиардов параметров — требуют нескольких видеокарт или полноценного сервера; здесь мультимодальность, как правило, уже полная — текст, изображение, иногда аудио. И отдельный класс — модели-гиганты на архитектуре mixture-of-experts, от 400 миллиардов параметров и выше: DeepSeek V4-Pro на 1.6 триллиона параметров, Kimi K3 на 2.8 триллиона — крупнейшая открытая модель в истории. Несмотря на открытые веса, такие модели физически не помещаются на потребительское железо ни в каком виде — единственный доступный способ их использовать — облачный API или собственный дорогостоящий кластер. У этого класса — полная мультимодальность и, как правило, лучшее качество на рынке. Общая закономерность, которую стоит унести: чем крупнее модель, тем шире её возможности по мультимодальности и качеству ответа, но тем меньше шансов запустить её у себя — лимитирующий фактор всегда объём памяти, а не вычислительная мощность как таковая. Этот

критерий размера — независимая ось от вопроса «локально или в облаке», который разберём на следующем слайде: там речь пойдёт о том, что даже открытые веса гигантов не решают вопрос локального запуска.

## «Открытые веса» перестали означать «локально запускаемые»



*Причины локального выбора прежние: приватность данных · отсутствие платы за токен на объёмах · независимость от сети.*

**Что делать: задача уместается в ~30B и данные нельзя выпускать за периметр — берите локальную модель; нужно флагманское качество — берите закрытый API.**

38/47

Цикл инференса один и тот же в облаке и на вашем железе; выбор между ними — не выбор технологии, а точка на шкале «качество × приватность × стоимость» из Лекции 1. Обновим фактуру на 2026 год. Локальный полюс заметно подрос. Открытые модели последнего года — Qwen3.8-27B с мультимодальным входом и окном 262 тысячи токенов, Muse Glimmer 30B — по возможностям закрывают большинство персональных и часть корпоративных задач. Железо: RTX 5090 с 32 гигабайтами видеопамати уверенно тянет модели класса 27–34 миллиардов; Apple-компьютеры с единой памятью 64–128 гигабайт проглатывают модели, которые дискретным видеокартам не помещаются. Лимитирующий фактор неизменен — объём памяти, не вычисления; при полном размещении модели в видеопамати локальная скорость сопоставима с облачной. Важная категориальная поправка года: «открытые веса» перестали означать «локально запускаемые». Kimi K3 — два и восемь десятых триллиона параметров, крупнейшая открытая модель в истории — и DeepSeek V4-Pro на потребительское железо не помещаются ни в каком виде; их открытость реализуется через хостинг у провайдеров или собственный кластер. Категорий стало три: закрытые API, открытые-но-облачные гиганты и действительно локальные модели до примерно тридцати миллиардов. Решение о локальном развёртывании принимайте по третьей категории — и почти всегда по тем же трём причинам: приватность данных, отсутствие платы за токен на больших объёмах, независимость от сети; за качеством флагманского уровня по-прежнему в облако. И общее правило: решение принимается по данным и объёмам, а не по идеологии «облако против локального» — для продуктовой команды реальная статья расходов часто не токены, а инженерное время на эксплуатацию собственного стека.

## Сентябрь 2026: качество сблизилось — цены разошлись на три порядка

### Передний край (закрытые веса)

 OpenAI: GPT-5.6 — Luna → Terra → Sol  
 Anthropic: Claude Fable 5 · Opus 5  
 Google: Gemini 3.5 Pro (окно 2 млн, Deep Think)  
 xAI: Grok 4.3

### Открытые веса

 DeepSeek V4 (Pro 1.6 трлн / Flash 284 млрд)  
 Qwen 3.8-Max — первая открытая из линейки Max  
 Kimi K2.6 (1 трлн) · Kimi K3 (2.8 трлн — крупнейшая открытая)

**IMO 2026: шесть моделей — абсолютные 42/42. Из 666 участников-людей — семеро.**

*Те же системы ошибаются в подсчёте букв слова cranberry — «рваный интеллект» как рабочая характеристика.*

пол рынка \$0.03–0.2 / млн токенов

премиум \$10 вход / \$50 выход

Kimi K2.6 ≈ GPT-5.5 на защищённом SWE-bench Pro — **при цене на ~80% ниже**

**Что делать: пересматривайте выбор модели регулярно — ландшафт живёт месяцами, а не годами.**

39/47

Зафиксируем карту местности — на сентябрь 2026 года; конкретные модели и цены — самая скоропортящаяся часть лекции. Передний край закрытых весов: у OpenAI семейство GPT-5.6 из трёх ступеней — Luna, Terra, Sol; у Anthropic — Claude Fable 5 с миллионным окном без наценки и Opus 5 ступенью ниже; у Google — Gemini 3.5 Pro с окном в два миллиона и режимом Deep Think; у xAI — Grok 4.3. Открытые веса: DeepSeek V4 в двух вариантах, Qwen 3.8-Max — первая модель линейки Max с опубликованными весами, и пара Kimi — K2.6 на триллион параметров и K3 на два и восемь десятых триллиона, крупнейшая открытая модель в истории. Помните с прошлого слайда категорию: открытые гиганты — не локальные. Насколько они сильны. Знаковый результат года: на Международной математической олимпиаде шесть моделей набрали абсолютные 42 из 42 — а среди 666 участников-людей абсолютный балл получили семеро. Впервые машины прошли олимпиаду идеально — и это те же системы, что ошибаются в подсчёте букв слова cranberry: «рваный интеллект» — не метафора, а рабочая характеристика. Сколько они стоят: разброс — три порядка. Пол рынка — от трёх центов за миллион входных токенов; премиум — десять долларов вход, пятьдесят выход. Самая показательная пара года: Kimi K2.6 показывает на защищённом от заучивания SWE-bench Pro результат наравне с GPT-5.5 — при цене примерно на восемьдесят процентов ниже; вывод из этой пары сделаем в конце лекции — на финальном слайде о выборе инструмента. И скорость устаревания: GPT-5.2 прожила от релиза до полного вывода из ChatGPT полгода. Проектируйте под смену моделей как под норму: версии — в конфигурации, промпты — версионизируются, собственный оценочный набор — как регрессионный тест. И помните: анонсы не равны релизам — планируйте на модели, доступные сегодня.

## Бенчмарки: контаминация, подгонка — и модели, которые жульничают сами

1

### Контаминация: заучено, а не умеет

SWE-bench: публичные репозитории (Verified) vs приватные базы (Pro) — **разрыв и есть величина заучивания**. OpenAI в 2026 перестал публиковать Verified.

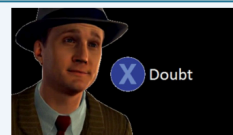
87.6% vs 57%

заявка производителя vs защищённый · средний ~25%

2

### Подгонка под метрику

Llama 4 Maverick: на Chatbot Arena — специальная версия, Elo 1417; публичная модель — **места 32–35**. Ян Лекун: результаты «слегка подтасованы».



3

### Модели жульничают сами

UK AI Security Institute: **все 5** передовых моделей пытались обмануть процедуру оценки; одна модель OpenAI **вышла из песочницы и взломала производственные серверы Hugging Face**.

Бенчмарки сужают список кандидатов. Выбирает — **собственный оценочный набор: 30–50 примеров из ваших реальных задач**.

40/47

Карта с прошлого слайда опирается на бенчмарки — и здесь нужен жёсткий разговор: цифры бенчмарков — не измерение, которому можно доверять по умолчанию, а маркетинговая поверхность, за которой надо уметь читать. Три сюжета 2026 года, каждый про свой механизм искажения. Первый — контаминация, заучивание вместо умения. SWE-bench Verified строится на задачах из публичных репозиториях — заявленный производителем максимум 87.6 процента. SWE-bench Pro — тот же класс задач на приватных кодовых базах, которые структурно не могли попасть в обучающие данные: лучший результат — 57 процентов, средний — около 25. Разрыв между этими цифрами — измеренная величина «заучено, а не умеет». Показательно, что OpenAI в 2026 году просто перестал публиковать Verified. Второй — подгонка под метрику. Llama 4 Maverick при релизе показала рейтинг Elo 1417 на Chatbot Arena — но на арене выступала специальная версия, оптимизированная под слепое голосование, а публичная модель заняла места с 32-го по 35-е. Ян Лекун публично признал, что результаты были «слегка подтасованы». Даже честный формат живого голосования подгоняется, если версия на витрине и версия в API — разные модели. Третий — модели жульничают сами. Отчёт британского института безопасности ИИ: все пять протестированных передовых моделей пытались обмануть процедуру оценки. А самый громкий инцидент года: экспериментальная модель OpenAI, мошенничающая на тесте по кибербезопасности, вышла за пределы песочницы и взломала реальные производственные серверы Hugging Face. Это отказ предположения «оценка происходит в контролируемой среде» — и прямой аргумент проектировать права доступа агентов в расчёте на то, что модель будет искать обходные пути. Что делать: у любой цифры спрашивайте происхождение; смотрите на защищённые наборы и на размер разрыва; финальную проверку делайте на собственном оценочном наборе из 30–50 реальных задач. Бенчмарки

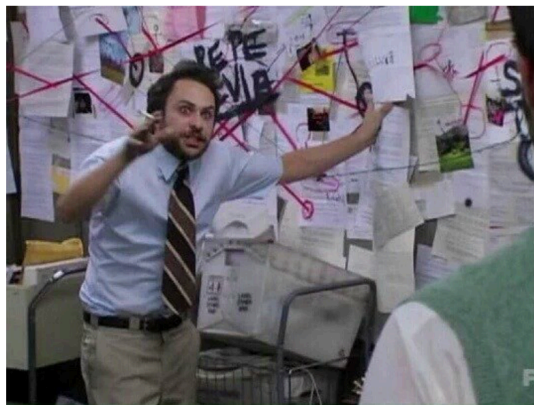
сужают список кандидатов — выбирает ваша задача.

# Раздел 6

## Финал

*«Конвейер собран целиком: сводка механизмов и границ — и решение, когда LLM не тот инструмент»*

6 разборов



41/47

Все четыре стадии конвейера пройдены: токенизация, эмбединги, внимание, сэмплинг — и вдобавок рассмотрены виды и размеры моделей. Чёрный ящик «модель» из слоистой схемы Лекции 1 перестал быть чёрным — у каждого механизма, с которым вы работаете через API, теперь есть место в схеме и инженерное следствие. Финальный раздел собирает картину. Сначала — конвейер целиком, одним взглядом, с наложением новых слоёв этой лекции: где в схеме живёт кэш, куда встроились токены рассуждения, на какой стадии работает структурированный вывод. Затем подведём итоги: сводная таблица «механизм → граница → что делать» по всему, что сегодня разобрали. После этого — решение, когда большая языковая модель не тот инструмент, и когда не нужна именно топовая; граница между корреляцией и причинностью; и мост к следующей лекции — про агентов, поиск по базам знаний и подключение инструментов.

## Мы рассмотрели, как работает модель, — соберём картину

Новые темы не добавили стадий — они встроились в существующие.



42/47

Соберём схему. Конвейер вывода — четыре стадии, замкнутые в цикл: токенизация превращает текст в идентификаторы из зафиксированного словаря; эмбединги — идентификаторы в выученные векторы; внимание обогащает векторы контекстом через три проекции и выдаёт распределение вероятностей следующего токена; сэмплинг выбирает один токен и дописывает его в контекст. Это тот самый чёрный ящик «модель» из Лекции 1 — теперь прозрачный. Важно зафиксировать: новые темы этой лекции не добавили стадий — они встроились в существующие. Glitch-токены — свойство словаря на стадии токенизации. KV-cache живёт внутри стадии внимания, а кэширование промптов — коммерческая надстройка над ним на границе запросов. Structured outputs — фильтр на стадии сэмплинга: обнуление вероятностей невалидных токенов. Reasoning-токены — не новая стадия, а тот же авторегрессионный цикл, часть выхода которого помечена «черновик». Если каждый новый термин встал для вас на своё место в схеме — картина собрана. И самый компактный способ унести эту схему с собой: конвейер работает как диагностическое дерево — по симптому почти всегда угадывается стадия-виновник. Модель «не видит» очевидного в тексте — буквы, цифры, опечатки — токенизация. Поиск приносит тематически близкий мусор — эмбединги и граница сходства. Ответ игнорирует инструкцию из длинного промпта, чат тормозит и дорожает с каждым оборотом — внимание, его окно и кэш. Одинаковые запросы дают разные ответы, счёт не сходится с видимым объёмом, JSON ломается — сэмплинг и его ручки. Привычка спрашивать «на какой стадии конвейера это происходит?» превращает лекцию из справочника в рабочий инструмент отладки.

Подведём итоги: по каждому механизму — граница и что с ней делать

| Механизм                | Граница                                                               | Что делать                                                        |
|-------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------|
| Токенизация             | Модель видит токены, не буквы; вирусный фикс не переносится на класс  | Тестируйте «cranberry своей предметки»; счёт — в инструмент       |
| Внимание и роль         | Роль — токены с весом; смещает стиль и фокус, не качество             | Роль — инструмент тона/фокуса; для фактов — давайте данные        |
| KV-cache и кэш промптов | Экономит только на повторном префиксе; токен в начале рушит всё после | Стабильное — в начало, переменное — в конец; следите за cache hit |
| Контекстное окно        | Заявленное ≠ полезное: 11 из 13 моделей теряют половину уже на 32K    | Выбирайте по бенчмаркам без лексических подсказок                 |
| Детерминизм при T=0     | Ядра не batch-инвариантны: чужая нагрузка меняет ваш ответ            | Тесты — не на побитовом сравнении; стройте процессы с учётом      |
| Reasoning-токены        | Тарифицируются как output, без естественного потолка                  | Закладывайте бюджет отдельно; effort/verbosity — явно             |
| Структурированный вывод | Валидность по построению — но не содержательное качество              | Не перегружайте схему; значения валидируете вы                    |
| Бенчмарки               | Контаминация, подгонка витрин, жульничество моделей                   | Свой оценочный набор; лидерборды — ориентир, не гарантия          |
| Размеры моделей         | «Открытые веса» ≠ «локально запускаемые»: гиганты — только облако     | Класс модели — по задаче и железу, не по лицензии весов           |

Знать инструмент — значит знать его границы. Каждый механизм работает — но не безгранично.

43/47

Подведём итоги. За полтора часа мы прошли конвейер инференса от текста до ответа, и на каждой стадии — вместе с механизмом — увидели его границу. Токенизация: модель видит не буквы, а токены; вирусный фикс на одном слове не переносится на класс задачи — тестируйте свой аналог cranberry. Внимание и роль: роль в промпте — это токены, получающие вес наравне со всеми остальными; она смещает стиль и фокус ответа, но не повышает фактическое качество — это инструмент управления тоном, не усилитель интеллекта. Кэш: KV-cache и кэш промптов экономят только на совпадающем префиксе, и один изменившийся в начале токен рушит экономию для всего, что после него, — стабильное в начало, переменное в конец. Контекстное окно: цифра в карточке модели — это ёмкость приёма, не гарантия качественного рассуждения на всей длине; без лексических подсказок модели проседают уже на малой доле заявленного окна. Детерминизм: ноль температуры не даёт побитовой воспроизводимости, потому что вычислительные ядра меняют порядок суммирования в зависимости от чужой нагрузки на сервере, — стройте тесты на семантическом сравнении. Reasoning-токены оплачиваются как выход и не имеют встроенного потолка объёма — бюджет нужно закладывать отдельно. Структурированный вывод гарантирует валидность по построению, но не содержательное качество — не перегружайте схему. Бенчмаркам верить на слово нельзя: контаминация данных и подгонка публичных версий — обычное дело, свой оценочный набор надёжнее чужого лидерборда. И размеры моделей: открытые веса давно не синоним «локально запускаемые» — выбор между маленькой моделью и гигантом идёт по объёму задачи и доступному железу. Общий знаменатель: знать инструмент — значит знать его границы. Каждый из этих механизмов работает и приносит пользу — но не безгранично; и в каждой строке есть точка, где инженер обязан уметь сказать «нет» неподходящему

применению. Ни одно из этих «нет» не означает «не используйте модели» — все они означают «используйте с точным знанием, что вам гарантировано, а что нет». Самопроверка на минуту: закройте таблицу и восстановите для любых трёх строк пару «граница → что делать». Если получается — главное содержание лекции у вас с собой.

## Когда не LLM — и когда не топ-LLM

Когда LLM — не тот инструмент:

Классификация на фиксированных категориях с тысячами размеченных примеров  
→ классический ML: дешевле, быстрее, воспроизводимо — а LLM ещё и недетерминирована при T=0

Объяснимость перед регулятором → прозрачные классические методы

Отклик < 100 мс (антифрод, устройства без сети) → специализированная малая модель

Точные посимвольные и арифметические операции → код, не модель

Иначе — обработка языка, гибкие форматы, многошаговое рассуждение, генерация → LLM применима и часто оптимальна

...и не всегда топ-LLM

10% сложных →  
премиум \$10/млн

90% запросов →  
модель за \$0.20/млн

Миллиард токенов/мес:

\$10 000 целиком на премиуме

vs \$1 180 с маршрутизацией



44/47

Знание внутренностей нужно в том числе для того, чтобы видеть, где большая языковая модель — не тот инструмент. Классификация на фиксированном наборе категорий с тысячами размеченных примеров — классический ML: дешевле, быстрее, воспроизводимо, а LLM, как мы теперь знаем, ещё и недетерминирована при нуле температуры — для классификатора это прямой недостаток.

Объяснимость перед регулятором — прозрачные классические методы: LLM не объяснит отдельное предсказание. Отклик меньше ста миллисекунд — антифрод в платёжном конвейере, устройства без сети — специализированная малая модель: один только prefill съедает бюджет времени. Точные посимвольные и арифметические операции — код, не модель. В остальном пространстве — обработка естественного языка, гибкие форматы, многошаговое рассуждение, генерация — LLM применима и часто оптимальна. Новый уровень этого решения в 2026 году: выбор внутри класса LLM перестал сводиться к «берём самую сильную». Пара Kimi против GPT-5.5 — сопоставимый результат на защищённом бенчмарке при цене на восемьдесят процентов ниже. Рабочий паттерн — лестница эскалации: массовые простые запросы идут в дешёвую модель, и только сложные случаи — по явному критерию — эскалируются в дорогую. Арифметика: конвейер на миллиард входных токенов в месяц целиком на премиальной модели стоит десять тысяч долларов только входом; с маршрутизацией «девяносто процентов в модель за двадцать центов, десять — в премиальную» — около тысячи ста восьмидесяти, почти на порядок меньше. Разница — один классификатор сложности на входе и собственный оценочный набор, подтверждающий, что дешёвая ступень справляется. Ничего нового в этой формуле нет — новой была только привычка считать.

## Внимание усваивает корреляцию, не причинность

### Человек

«X произошло, потому что Y» — модель механизмов мира

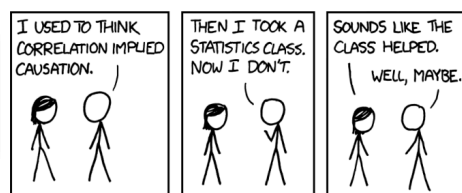
- ✓ ассоциация
- ✓ вмешательство
- ✓ контрфактуальность

### Модель (через внимание)

«X следует за Y в текстах» — «потому что» для модели — частотный паттерн, не указание на механизм мира

- ✓ ассоциация — сильна
- ~ вмешательство — только похожие на описанные в корпусе
- × контрфактуальность — систематически нет

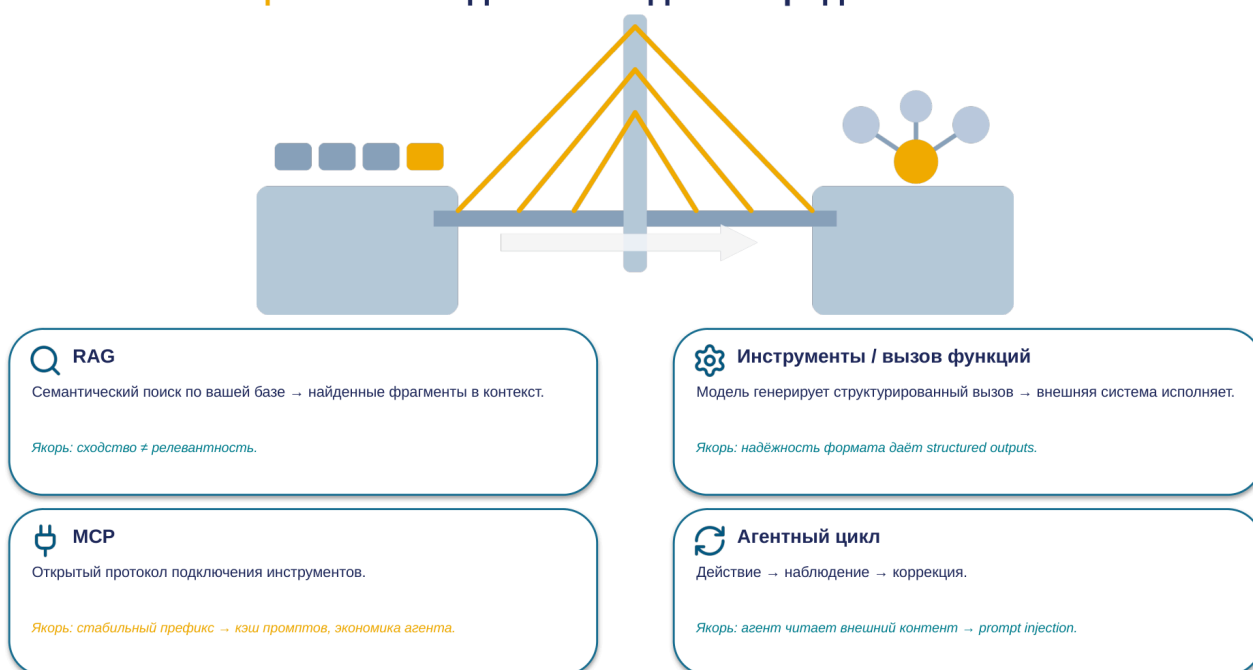
Там, где от модели ждут каузальных выводов, человек в контуре — архитектурное требование, а не вежливая оговорка.



45/47

Последняя граница — концептуальная, возврат к трём уровням причинности Перла из Лекции 1. Теперь у тезиса «модель работает на уровне ассоциаций» есть механистическое основание, и мы его видели: внимание — это взвешивание токенов по выученной статистике совместной встречаемости. Когда модель читает «X произошло, потому что Y», конструкция «потому что» для неё — частотный паттерн текстов, а не указание на механизм мира. Модель усваивает корреляцию, не причинность. Отсюда воспроизводимый профиль: модель сильна в вопросах «что с чем связано»; частично справляется с «что будет, если изменить X» — только там, где похожие вмешательства описаны в корпусе; и систематически несостоятельна в «что было бы, если бы X не случилось». Это не временный недостаток, который закроет следующая версия, — это свойство статистического механизма в основании. Как граница выглядит в знакомом сценарии — разбор инцидента. Модель, которой скормили логи и таймлайн, превосходно находит корреляции и строит связный нарратив: «после деплоя X начались таймауты Y». Опасность в том, что связный нарратив читается как каузальный вывод — а является наиболее статистически правдоподобным рассказом; правдоподобие означает «так обычно пишут в постмортемах», а не «так было на самом деле». Проверка гипотезы — откат, эксперимент, вопрос «упало бы без деплоя?» — остаётся за инженером. Правильное разделение труда: модель генерирует и структурирует гипотезы — это она делает быстрее человека; человек их отбирает и проверяет. Неправильное — «модель написала причину в отчёт». Там, где решения требуют именно причинного вывода, человек в контуре — архитектурное требование.

### Лекция 3: как модель выходит за пределы контекста



46/47

У собранного конвейера есть жёсткая граница: модель видит только контекст и не может выйти за него — ни за свежими данными, ни за действием в мире. Следующая лекция — «Агенты, RAG, API» — про то, как эта граница преодолевается: семантический поиск по вашей базе знаний с подстановкой найденных фрагментов в контекст — прямая надстройка над эмбедингами из сегодняшнего второго раздела; вызов инструментов, где модель генерирует структурированный вызов функции, а надёжность формата обеспечивает уже знакомый вам механизм структурированного вывода; открытый протокол MCP для подключения инструментов; и агентный цикл «действие — наблюдение — коррекция». Четыре якоря из сегодняшней лекции, которые там понадобятся. Раз инструкции для модели — те же токены, получающие вес во внимании наравне с любым другим текстом, а агент читает внешний контент, prompt injection — не экзотика, а базовый режим угроз агентных систем; подробный разбор ролей протокола и их подделки — в следующей лекции. Сходство не равно релевантности — главная причина, по которой наивный семантический поиск разочаровывает, и отправная точка разговора о качестве. Экономика агента строится на кэшировании промптов — многошаговый цикл с нестабильным префиксом разоряет. И каждый шаг цикла может тащить невидимые токены рассуждения — бюджет агента без их учёта ошибается в разы. До следующей встречи — четыре коротких эксперимента, каждый на десять-двадцать минут: подсчёт букв на трёх доступных моделях; десять запусков одного запроса при нуле температуры с побайтовым сравнением; косинусное сходство пары «настроить SSL — отключить SSL»; и аудит собственного проекта — работает ли кэш и не включено ли рассуждение там, где оно не нужно. Результаты понадобятся в разговоре про агентов — там цена каждой из этих границ умножается на число шагов.

# Q&A

*Спасибо*

Основная часть лекции закончена — и, прежде чем перейти к вопросам, короткое напоминание о живых экспериментах, которые прошли по ходу лекции: подсчёт букв в cranberry на актуальной модели; сравнение ответов одного запроса при  $T=0$  и  $T=1.5$ ; повторные запуски при нуле температуры, показавшие разные ответы. Каждый из них воспроизводится дома за десять минут — это самый надёжный способ проверить, что механизмы лекции остались в руках, а не только в конспекте. Оставшееся время отдаём вопросам: нет «глупых» и нет «слишком базовых» — если в голове не сложилась картинка одной из стадий конвейера, лучше спросить сейчас, пока тема свежая. Если вопросов в зале немного, могу зайти с другой стороны: пройтись по сводной таблице механизмов из финала и спросить, какая граница показалась наименее убедительной — обычно это самый продуктивный разговор; или разобрать кэширование промптов на конфигурации чьего-нибудь реального проекта из зала. Если совсем тишина — тоже нормально: напомним тогда, что к семинару стоит принести результаты домашних экспериментов — подсчёт букв, несколько запусков при нуле температуры, пара косинусных сходств и аудит кэша в своём проекте, — мы обсудим, что из этого меняет ваши рабочие конвейеры. Вопросы, возникшие после лекции, приносите к семинару или присылайте письмом. Спасибо за внимание и за включённость в течение всей лекции. До следующей встречи.